

Neuraal

Documento de presentación: Trabajo Final de Máster

Proyecto: Neuraal — Productivity App

Tipo: Trabajo Final de Máster (TFM)

Despliegue en producción: <https://neuraal.app>

Repositorio: <https://github.com/nikeon6/app-neuraal>

Autor: Sergio Caballero Entrena

Tutor/a: Mouredev

Máster: Máster en Desarrollo con IA – BIG School

Fecha de presentación: 23/02/2026

Índice:

1.	Resumen ejecutivo	03
2.	Motivación, origen del proyecto y pruebas de concepto	04
3.	Objetivos del proyecto	08
4.	Funcionalidades destacadas	09
5.	Stack tecnológico y justificación	25
6.	Arquitectura del sistema	28
7.	Modelo de datos	31
8.	Seguridad	32
9.	Inteligencia Artificial integrada	33
10.	Observabilidad y monitorización	35
11.	Calidad del software	36
12.	Despliegue y CI/CD	37
13.	Reflexión sobre el desarrollo	39
14.	Líneas futuras	40
15.	Conclusiones	41

1. Resumen ejecutivo

Neuraal es una aplicación web full-stack de productividad personal que permite a usuarios autenticados gestionar tareas, notas y temas a través de un dashboard interactivo orientado al calendario.

A diferencia de las herramientas de productividad convencionales, Neuraal integra funcionalidades de **inteligencia artificial** (resúmenes automáticos, clasificación de temas por embeddings, OCR y transcripción de vídeos de YouTube), un **editor de texto enriquecido** profesional, un sistema visual de **temas flotantes** con conexiones SVG, **recordatorios programados**, y un **resumen semanal** con gráficos analíticos.

El proyecto ha sido desarrollado siguiendo prácticas de ingeniería de software de nivel profesional: **Clean Architecture**, **TDD**, **CI/CD completo** con rollback automático, **observabilidad** (Sentry, Prometheus, logging estructurado, LangSmith), **seguridad por diseño** (OWASP Top 10) y **documentación exhaustiva** (16 ADRs, especificación OpenAPI, guías operacionales).

La aplicación está desplegada y funcionando en producción: <https://neuraal.app>

Es un proyecto open-source, y el repositorio es público: <https://github.com/nikeon6/app-neuraal>

2. Motivación, origen del proyecto y pruebas de concepto.

2.1 Motivación y origen del proyecto

Neuraal nace directamente de una necesidad práctica descubierta durante el estudio teórico del máster. Durante esta etapa, empecé a utilizar herramientas de productividad como **Notion** y **Obsidian** para organizar los apuntes, las tareas y los proyectos a realizar. Habitualmente utilizaba **Google Keep** para capturar notas rápidas, y uno de los puntos destacables para mí era **la sencillez y su organización intuitiva** que permite apuntar algo de manera directa sin perderte entre otras opciones.

Cuando probé Notion y Obsidian, me sorprendió el grado de personalización alto que permite montar cuadros de mando de manera muy visual. Sin embargo, este recurso conlleva un coste: **tiempo de configuración**, necesidad de **definir estructuras** (bases de datos, vistas, relaciones...) y una **curva de aprendizaje** que puede abrumar al inicio, sobretodo a usuarios sin experiencia trabajando con datos.

A partir de ahí surgió la idea original del proyecto:

- Construir una aplicación con la inmediatez y “fricción mínima” de Google Keep.
- Mantener la opción de escalar a un sistema más avanzado para organizar, clasificar y visualizar ideas y proyectos de forma agradable y práctica.
- Añadir componentes capaces de almacenar distintos tipos de contenido (tareas, notas, adjuntos... etc.)
- Integrar herramientas de IA que ayuden a clasificar, resumir y, en general, reducir el esfuerzo manual de organización.

En resumen, Neuraal pretende unir **simplicidad, rapidez, potencia y adaptabilidad** en una misma herramienta, permitiendo que el usuario pueda empezar a utilizarlo en segundos, pero también tenga margen para convertir su espacio en un sistema de trabajo más completo y visual.

2.2 Pruebas de concepto

El primer paso fue la comprobación de **la viabilidad del concepto**, tanto a **nivel técnico** como a **nivel de usabilidad**. Se desarrollaron varias pruebas de concepto, todas ellas sirvieron para comprobar si la propuesta podía implementarse de manera realista y ofrecer una experiencia de usuario positiva y coherente.

Para acelerar el proceso, se optó por un enfoque de prototipado rápido apoyado en el uso de **herramientas de IA** para explorar alternativas, resolver dudas de implementación y evaluar la viabilidad de determinados componentes (interacción, representación visual y comportamiento responsive). Gracias a este proceso, fue posible **probar distintas soluciones en poco tiempo**, descartar aquellas que no aportaban valor o no eran sostenibles, y reforzar las que sí encajaban con los objetivos fijados para este proyecto.

Con los resultados obtenidos, el diseño fue evolucionando de manera gradual: se realizaron ajustes en la distribución de la interfaz, en la presentación visual y en la interacción, **teniendo en cuenta tanto las posibilidades como las limitaciones detectadas** durante las pruebas mencionadas (rendimiento, complejidad visual, escalabilidad de layout y claridad para el usuario). Este proceso de **iteración continua** permitió converger progresivamente hacia una interfaz cada vez más parecida a la actual, añadiendo cantidad de cambios durante el proceso.

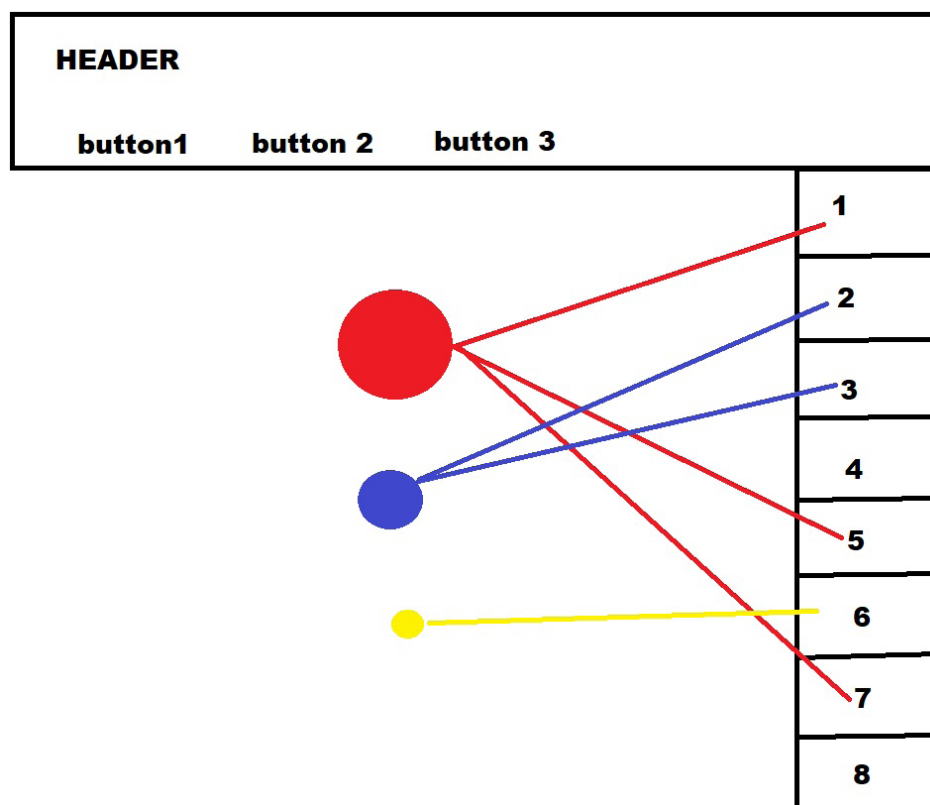


Figura 2.2.1 — Boceto inicial de distribución y conexiones

Una de las ideas diferenciales era **representar las temáticas como nodos (bolas)** y relacionarlas con tareas/elementos del día mediante conexiones visuales

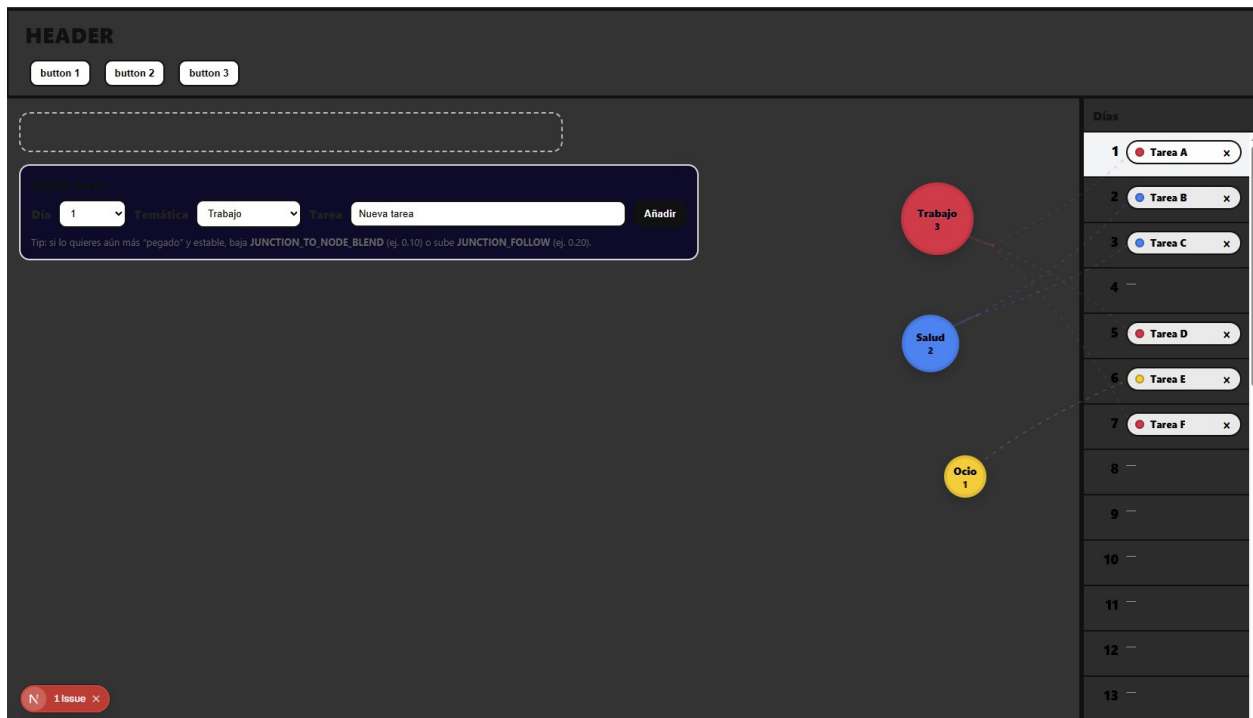


Figura 2.2.3 — Prueba de interfaz “diario / dashboard” y enfoque minimalista.

En paralelo, exploré un modo de vista más limpio tipo “Daily Log”, orientado a capturar notas o eventos con mínima fricción.

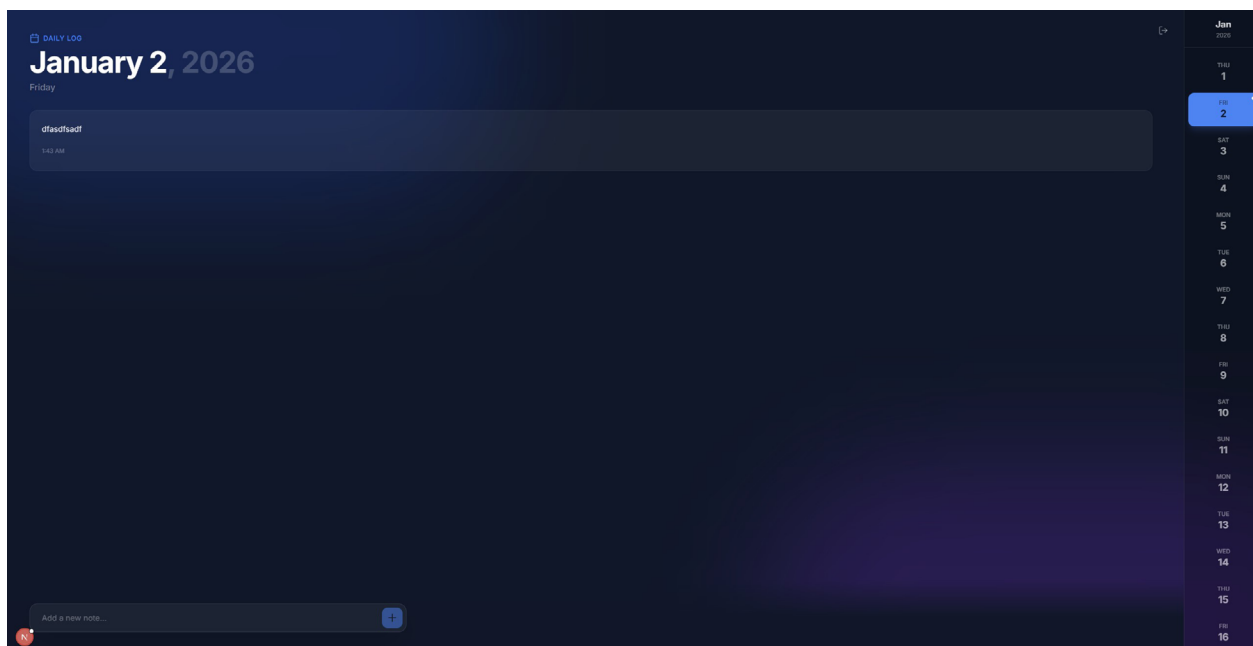


Figura 2.2.4 — Ajustes visuales: paleta de colores y coherencia estética.

Se probaron varias **paletas de colores, tipografías y ajustes visuales**, aunque al final se volvió al diseño legacy. En conjunto, **estas pruebas de concepto fueron clave para transformar la idea inicial en un diseño implementable**. No se trató de una única prueba, sino de un proceso iterativo en el que cada prototipo aportó información práctica: qué era viable, qué resultaba confuso para el usuario y qué mejoras eran necesarias.

Como resultado, se estableció una base sólida sobre la que continuar el desarrollo, llegando a una interfaz cercana a la actual y mejorándola de forma continua a lo largo del proyecto.

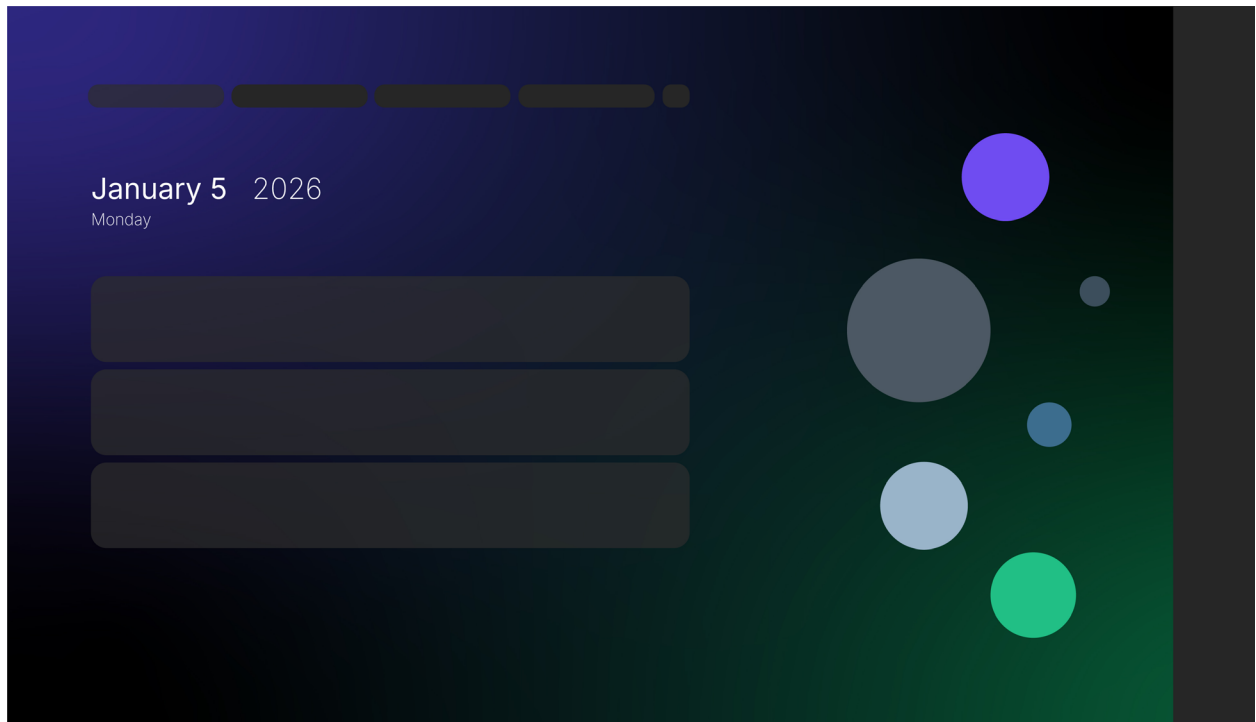


Figura 2.2.5 — Ajustes visuales: paleta de colores y coherencia estética.

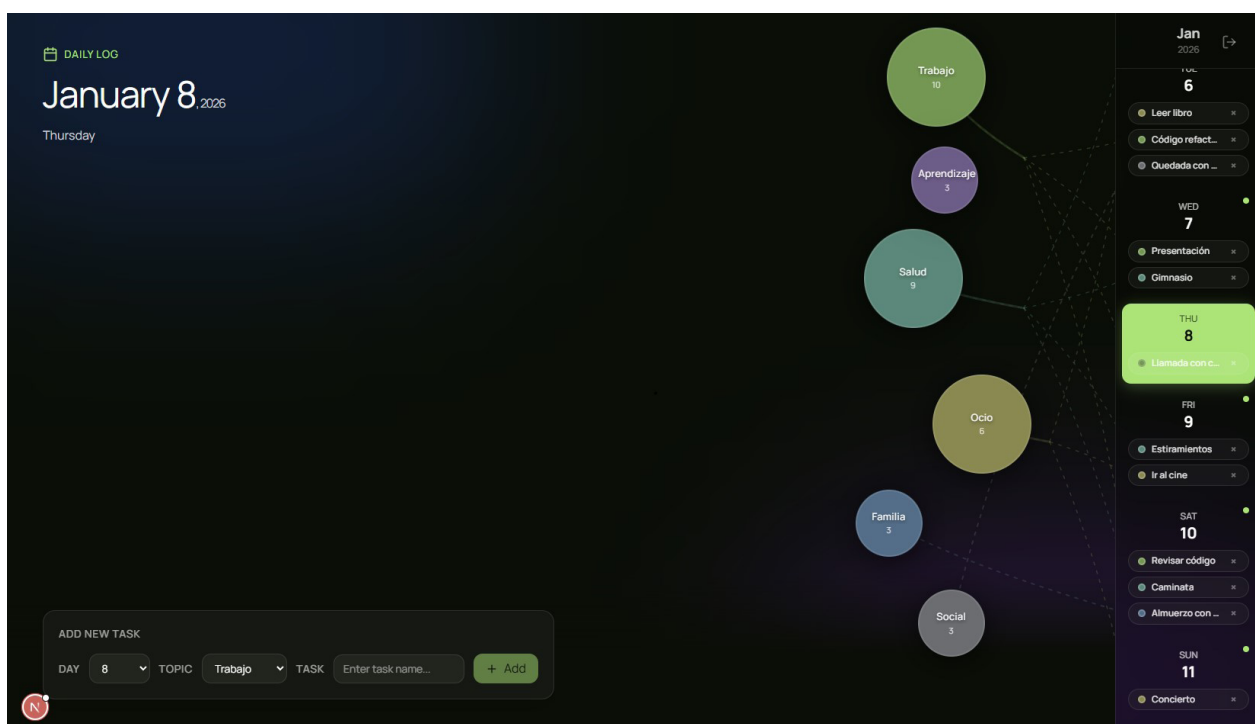


Figura 2.2.6 — Ajustes visuales: paleta de colores y coherencia estética.

3. Objetivos del proyecto

1. **Desarrollar una aplicación web full-stack** de productividad personal con autenticación segura, gestión de tareas/notas y organización temática.
2. **Integrar funcionalidades de IA** que aporten valor real al usuario: resúmenes automáticos, clasificación inteligente de temas, extracción de texto de imágenes y transcripción de vídeos.
3. **Aplicar Clean Architecture** con separación estricta de capas para garantizar la mantenibilidad y testabilidad del código.
4. **Implementar un pipeline CI/CD completo** que incluya testing automatizado, build Docker, despliegue y rollback automático.
5. **Desplegar en producción** la aplicación en un dominio público accesible (<https://neuraal.app>).
6. **Diseñar una interfaz original e innovadora** con elementos visuales diferenciadores (temas flotantes, calendario vertical, conexiones SVG).
7. **Documentar las decisiones arquitectónicas** mediante ADRs (Architecture Decision Records) y mantener documentación operacional.
8. **Garantizar la seguridad** siguiendo las directrices del OWASP Top 10.
9. **Implementar observabilidad completa:** logging estructurado, métricas Prometheus, seguimiento y versionado de prompts con LangSmith, error tracking con Sentry, y health checks.

4. Funcionalidades destacadas

4.1 Editor de texto enriquecido (TipTap)

El corazón de la aplicación es un editor de texto basado en **TipTap 3** (sobre ProseMirror), que ofrece:

- **Formato completo:** negrita, cursiva, subrayado, listas, encabezados.
- **Bloques de código** con syntax highlighting (lowlight) para múltiples lenguajes de programación.
- **Inserción de imágenes** tanto por URL como por subida directa al almacenamiento S3, con botón de escaneo OCR.
- **Incrustación de vídeos de YouTube** con botón de transcripción integrado.
- **Enlace de URLs** con detección automática.
- **Adjuntos** se pueden insertar en cualquier parte del contenido del editor, permite arrastrar y soltar archivos, imágenes y documentos o copiar y pegar, además de mostrar un resumen en la parte inferior de todo el contenido adjunto.
- **Auto-guardado** con debounce de 1800ms y control de concurrencia optimista (campo version): si dos pestañas editan la misma entrada, el servidor detecta el conflicto y solicita refresco.
- Selección de tipo de entrada: **Task o Note**
- Las **Tasks** se podrán marcar como completadas (por defecto pendientes) y las Notes no tendrán estado

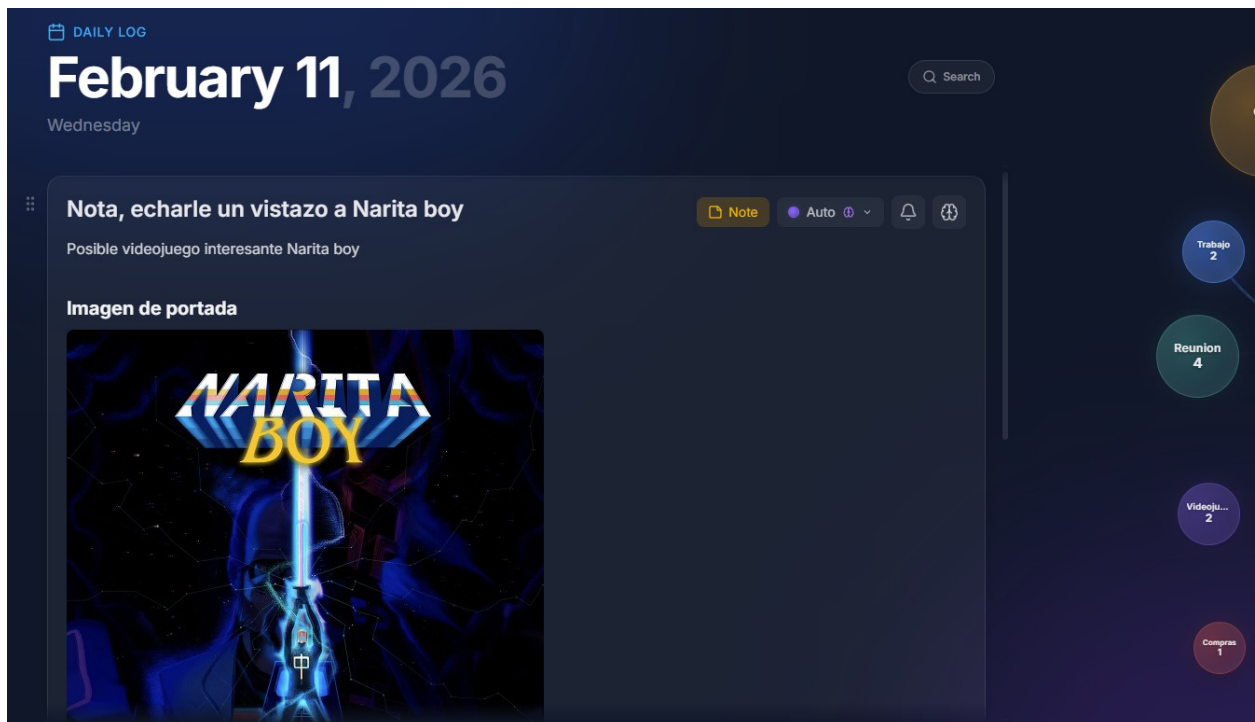


Figura 4.1.1 — Imagen adjunta.

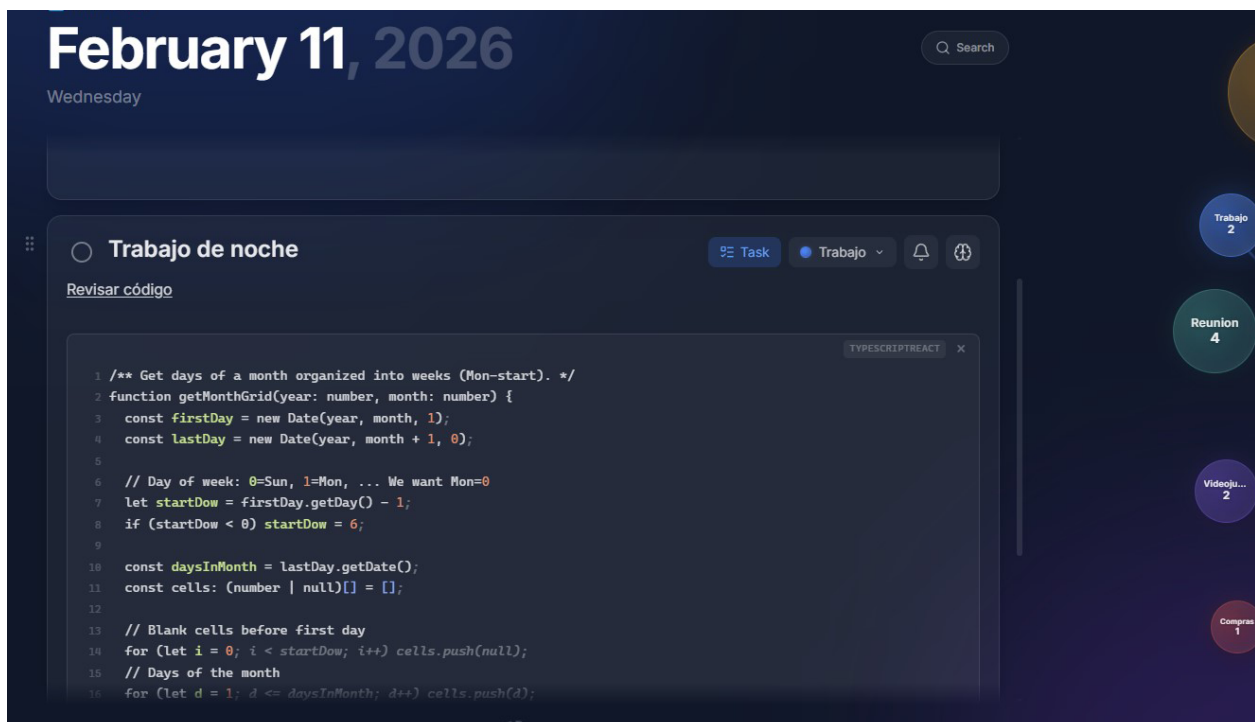


Figura 4.1.2 — Snippet de código



Figura 4.1.3 — Video de Youtube embebido

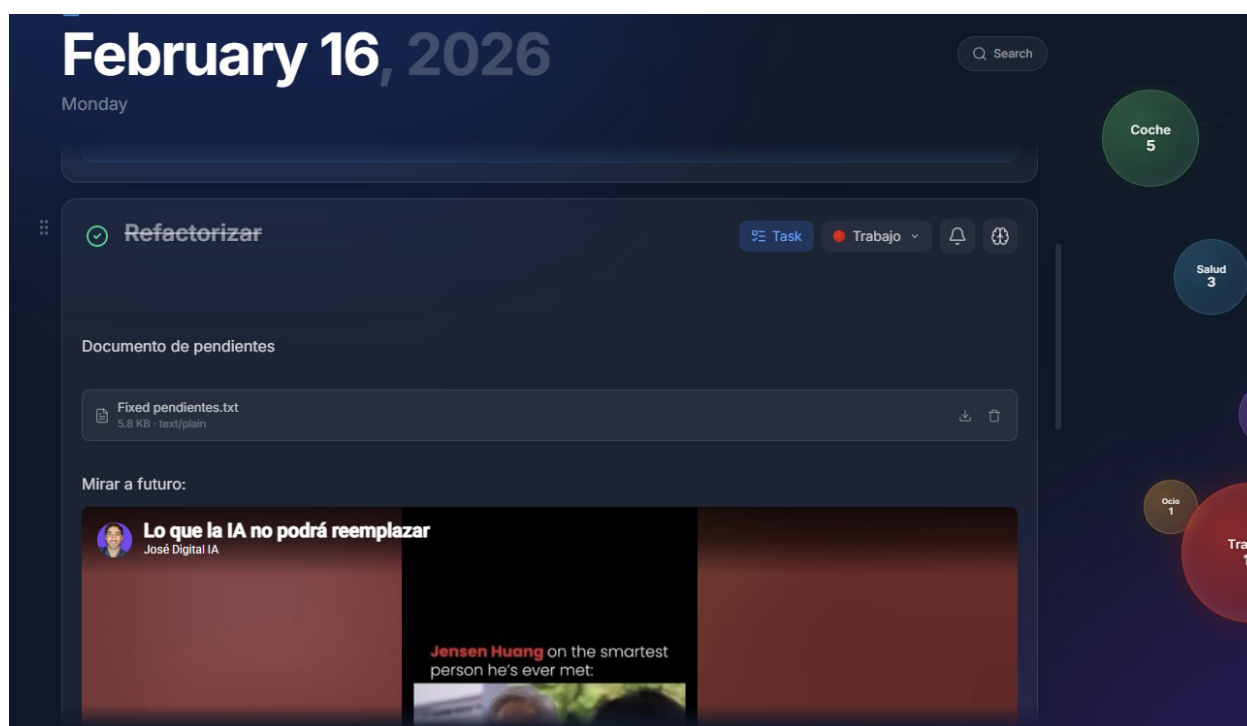


Figura 4.1.4 — Documento adjunto.

4.2 Burbujas flotantes “Neuronas”

Uno de los elementos visuales más diferenciadores de Neuraal:

- Los temas del usuario se muestran como **burbujas flotantes arrastrables** en pantalla.
- Las posiciones de las burbujas se guardan por usuario en localStorage y el usuario puede organizarlas (arrastrar) a su gusto.
- Por defecto los temas apuntarán mediante **conexiones SVG** (ramas animadas) a los días en el calendario que contengan notas/tareas de esta temática.
- Si colocamos el ratón encima de una burbuja se quedarán resaltadas solo las ramas de esta burbuja.
- Al seleccionar un tema (pinchar una burbuja), se mostrarán mediante pills las entradas relacionadas con el tema seleccionado en los días del calendario que los contengan, y se muestran conexiones SVG (ramas animadas) que unen visualmente la burbuja con todas las entradas de esta temática, se pueden seleccionar varios temas (burbujas) a la vez.
- La visualización se adapta de un layout vertical (escritorio) a una tira horizontal (móvil).

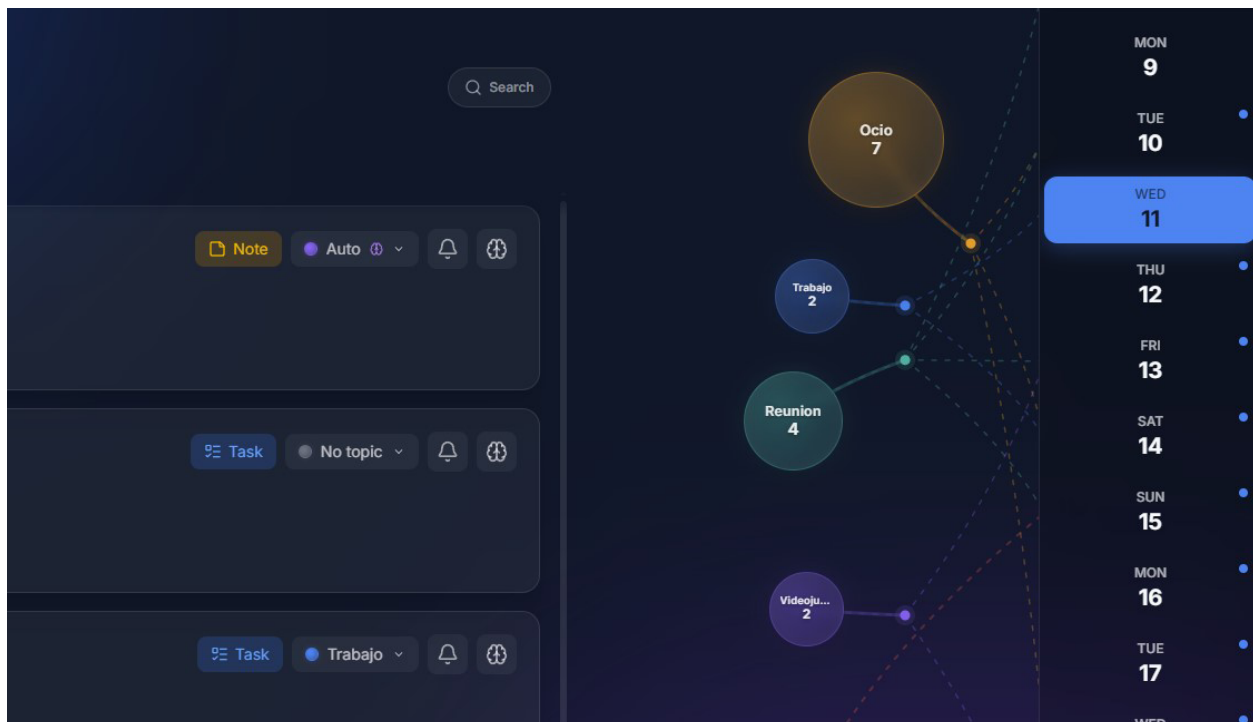


Figura 4.2.1 — Burbujas

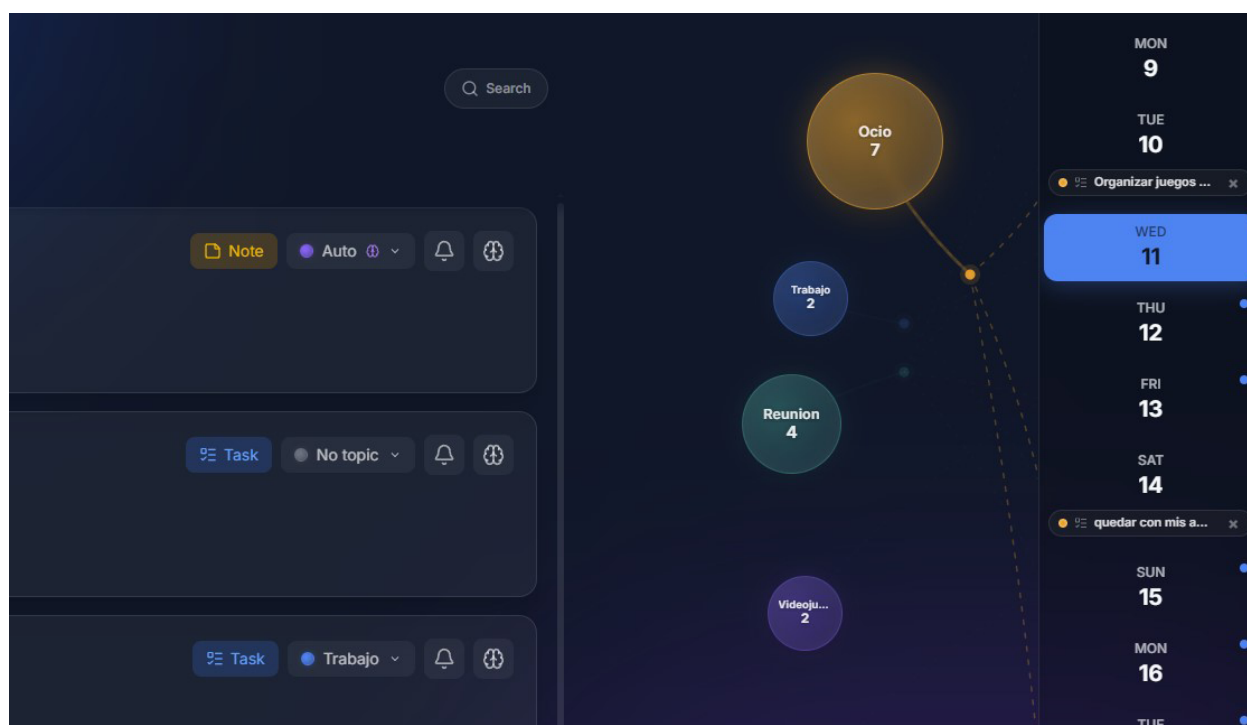


Figura 4.2.2 — Despliegue 1

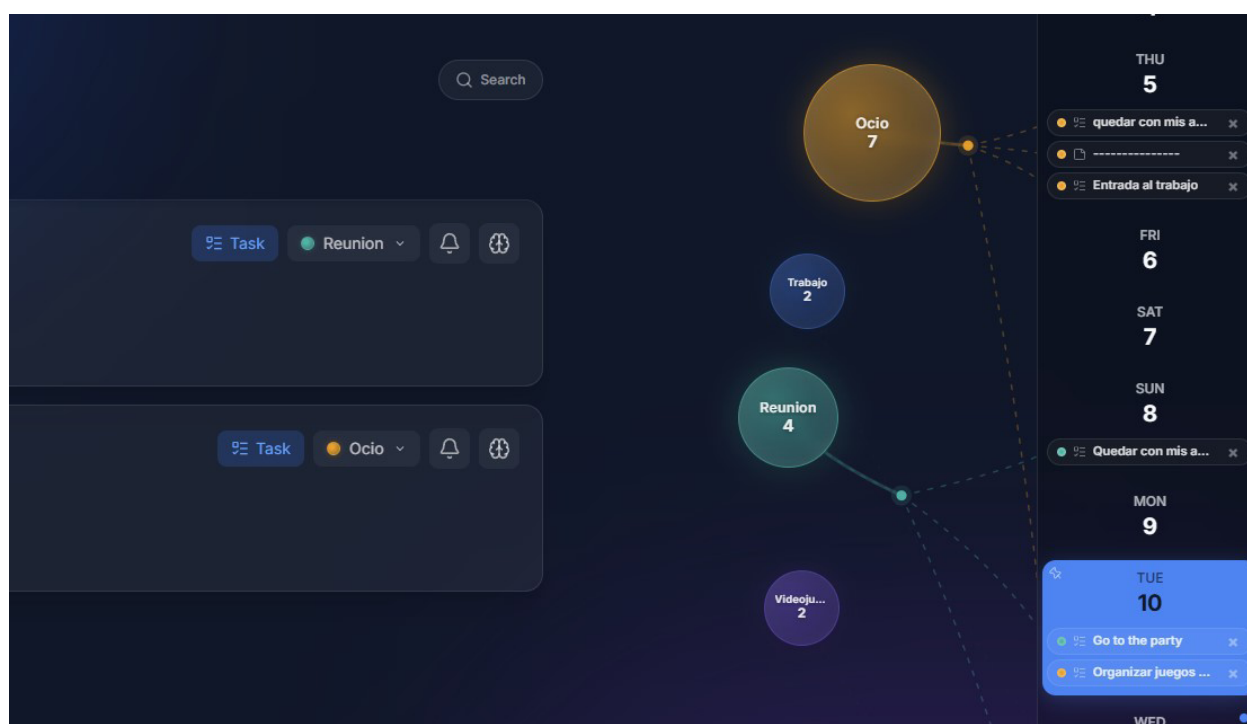


Figura 4.2.3 — Despliegue 2

4.3 Calendario vertical y navegación por día

- En **escritorio**: calendario vertical en el lateral derecho con indicadores de tareas por día (pills de colores), expandible para ver entradas por día.
- Al pinchar en un día, el día se desplegará en el calendario y mostrará todas las entradas de ese día mediante pills, esto provoca que se activen todas las burbujas (“neuronas”) de las temáticas contenidas en las entradas de ese día, por lo que al activarse también se visualizarán en el calendario en que otros días existen entradas de la misma temática que en el día seleccionado.
- Es posible pinear el día mediante un botón por si queremos mantener abiertos varios días a la vez en el calendario. (Por defecto, si cambiamos de día, el día anterior de repliega y oculta las entradas)
- En **móvil**: calendario horizontal desplazable con scroll táctil, indicadores de puntos y botones de navegación de mes en ambos extremos.
- Hacer clic en un día carga las entradas correspondientes en el panel principal.

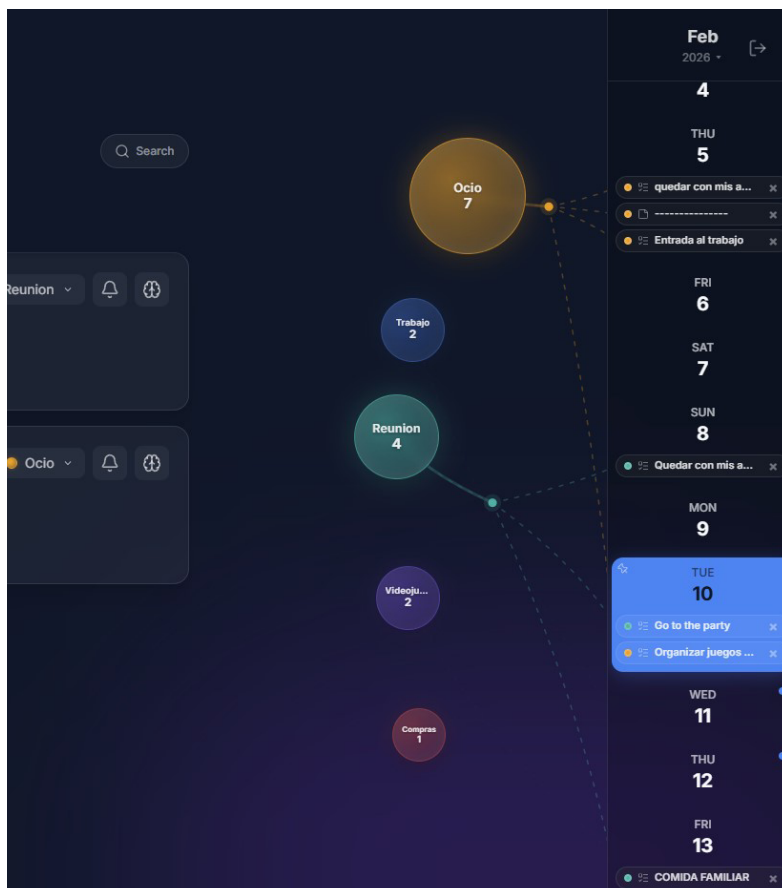


Figura 4.3.1 — Vista escritorio

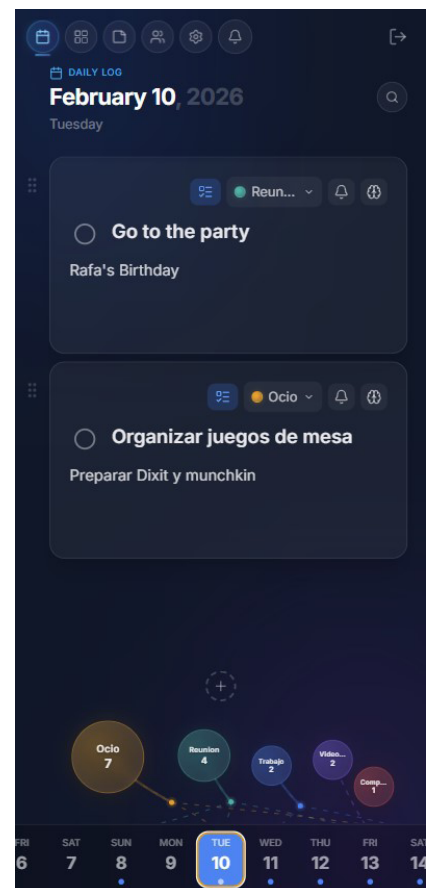


Figura 4.3.2 — Vista móvil

4.4 Drag-and-drop de tareas y notas adhesivas

- Las tareas se pueden **reordenar arrastrando** mediante Framer Motion Reorder, con un handle dedicado para evitar conflictos con el editor.
- Las **notas adhesivas** (stickies) también soportan drag-and-drop y se muestran en un layout kanban de dos columnas.

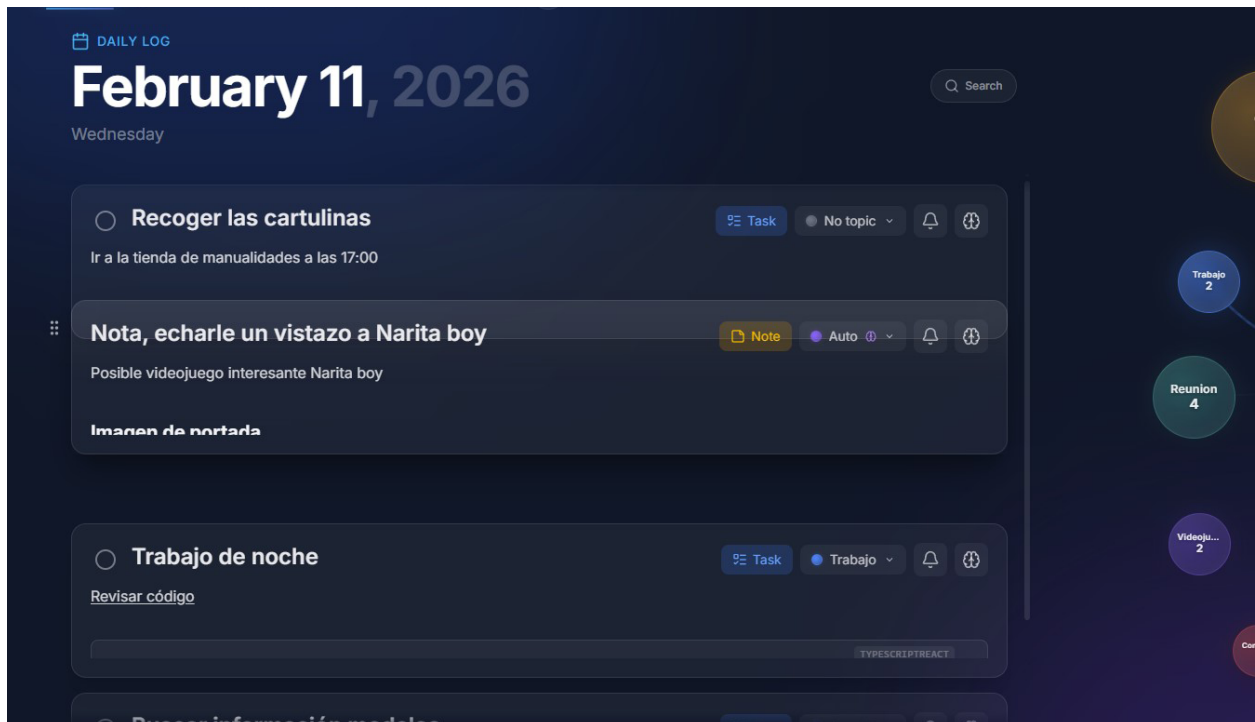


Figura 4.4.1 — Drag and drop

4.5 Recordatorios programados

- El usuario puede programar un recordatorio para cualquier tarea con fecha y hora específicas.
- Los recordatorios se pueden programar vía 3 canales: Push, Email y Whatsapp
- El flujo es completamente asíncrono:
 - La API encola el recordatorio en **BullMQ** (Redis).
 - Un **worker** dedicado procesa el job en el momento programado.
 - El worker envía un webhook a **n8n** para la entrega.
 - n8n llama de vuelta a la API con una **firma HMAC** para confirmar el envío.
 - Se crea una **notificación in-app** que el usuario ve en tiempo real.
 - El botón de recordatorio se desmarca automáticamente mediante un watcher de notificaciones.
- El uso de N8n nos da la posibilidad de ampliar las funcionalidades y automatizaciones de nuestros flujos de una manera sencilla y flexible.

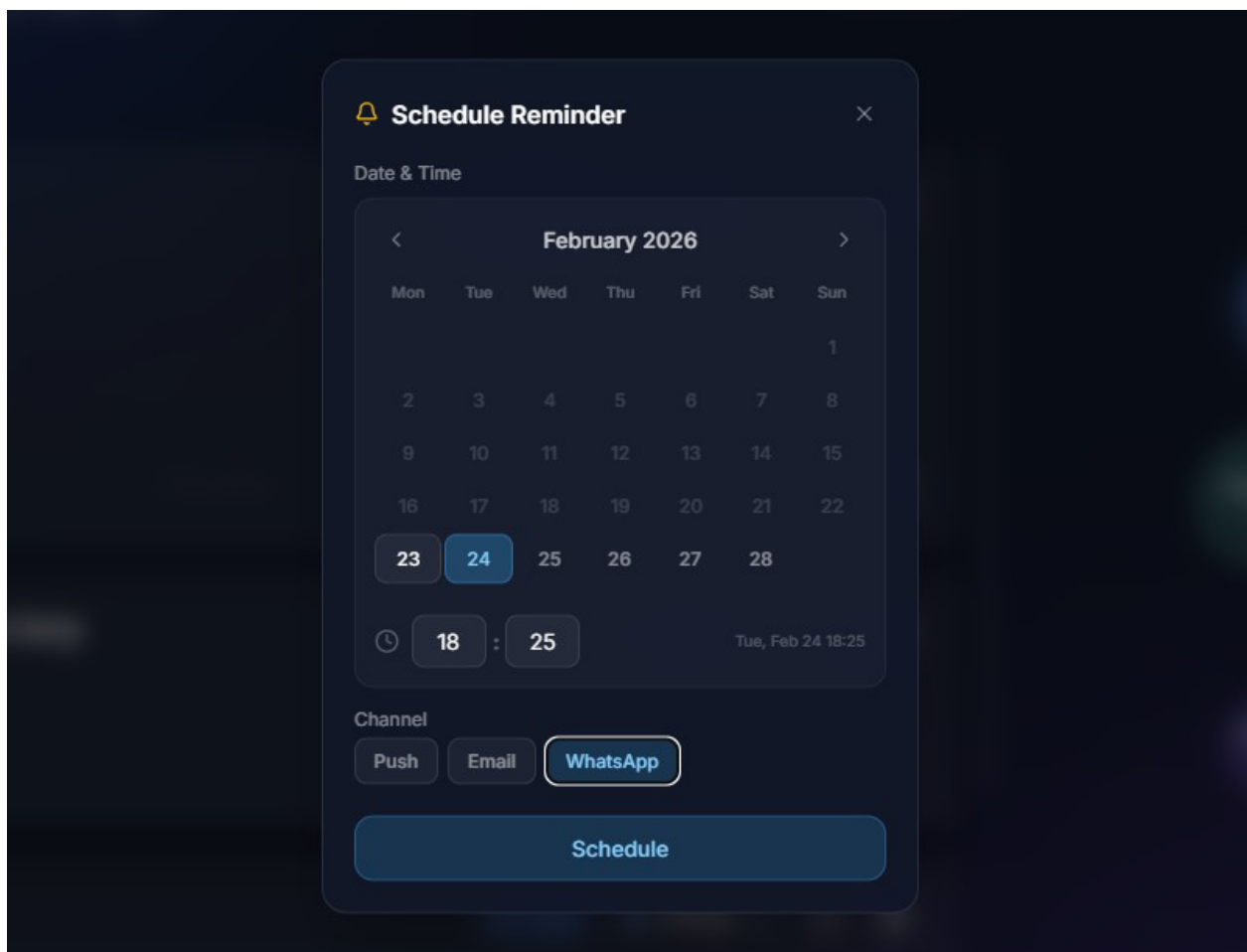


Figura 4.5.1 — Dialogo de recordatorio programado

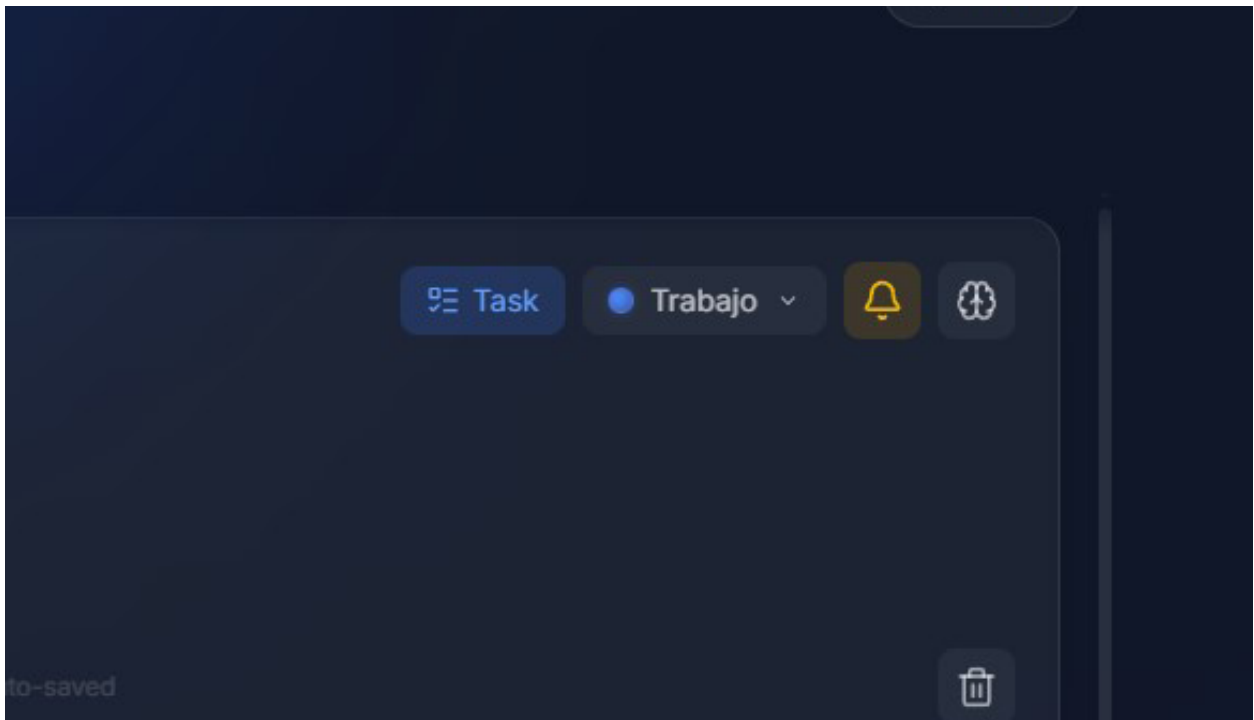


Figura 4.5.2 — Botón de recordatorio programado

4.6 Resúmenes automáticos con IA

- El usuario puede solicitar un **resumen** de cualquier entrada con un solo clic.
- Los resúmenes siempre quedarán visibles aún sin abrir la entrada, para de un vistazo rápido saber el contenido de la misma.
- El flujo es asíncrono: la API responde con 202 Accepted, un worker envía el contenido a n8n, n8n invoca un LLM, y el resultado vuelve mediante callback HMAC.
- El usuario recibe una notificación in-app cuando el resumen está listo y el contenido aparece debajo de la entrada.
- Protegido por **guardrails**: rate limiting por minuto/hora, control de concurrencia, cuota mensual y límite de tamaño de entrada.
- Gracias a usar n8n y este por detrás Langchain podemos hacer seguimiento de peticiones IA mediante LangSmith (observabilidad y versionado de prompts), al igual que en el futuro desarrollar fácilmente workflows de n8n más elaborados e inteligentes.

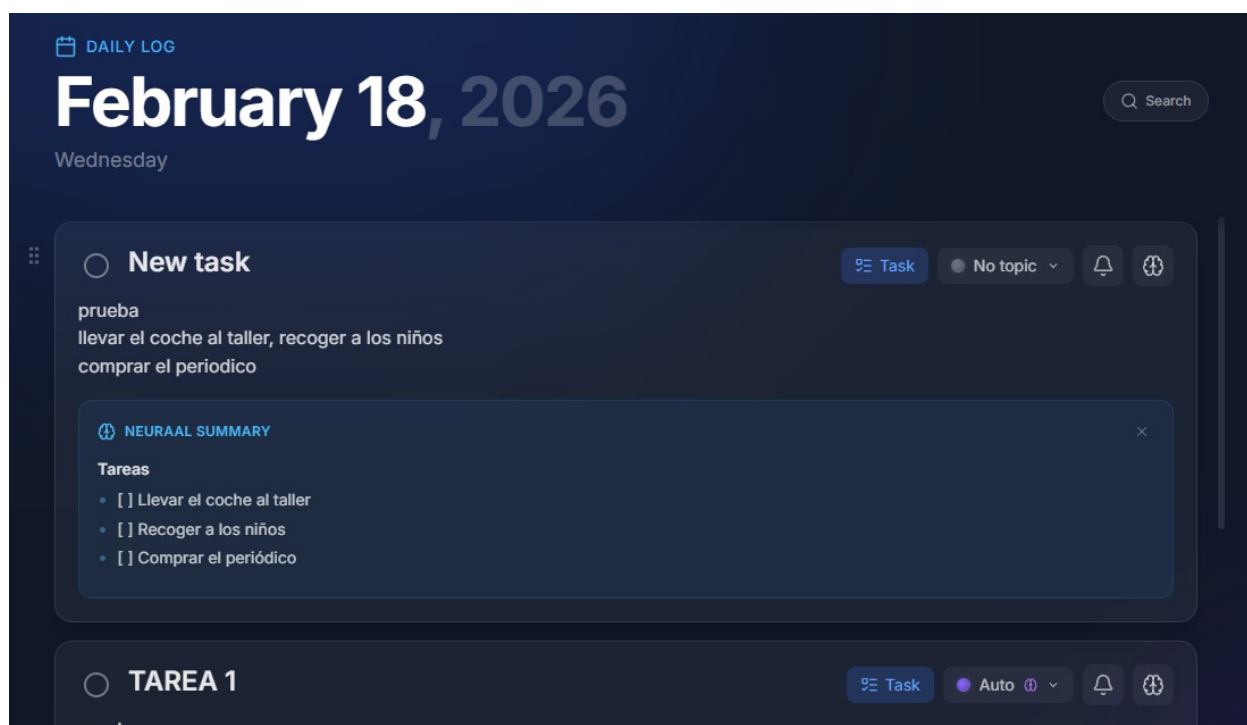


Figura 4.6.1 — Resumen automático

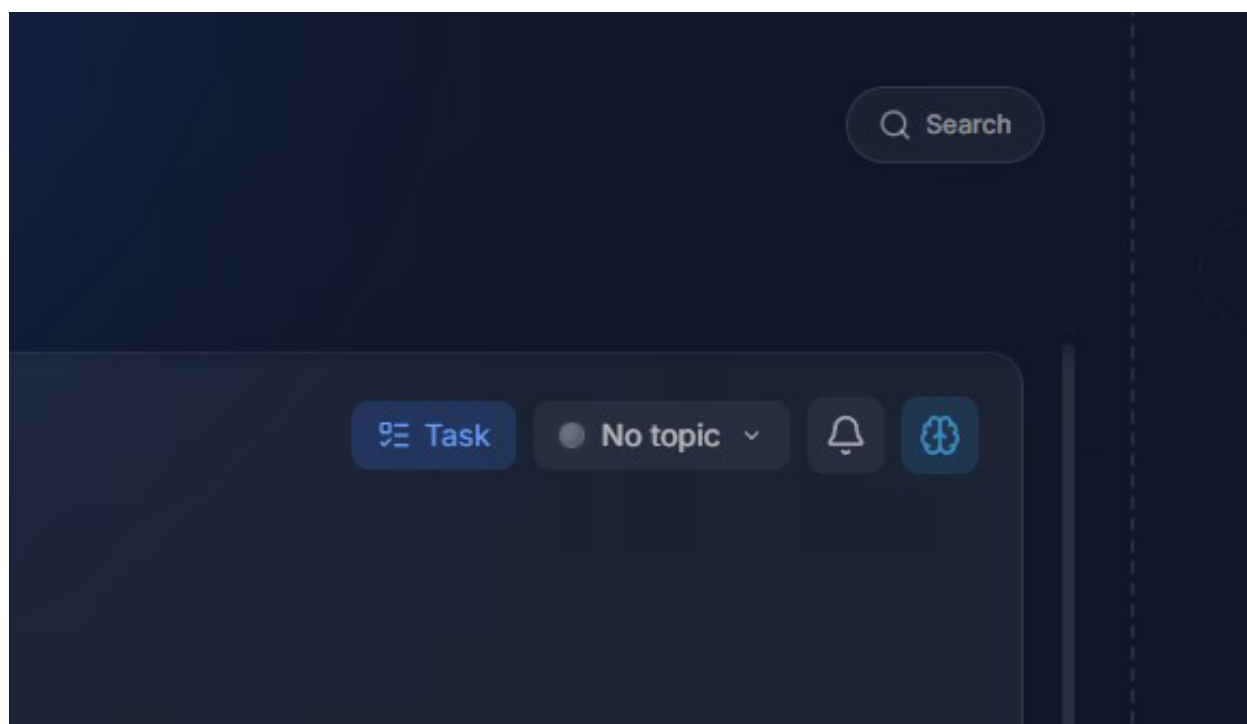


Figura 4.6.2 — Botón de resumen automático

4.7 Auto-clasificación de temas por embeddings

- Cada tema tiene un **embedding vectorial** de 4096 dimensiones generado por **Ollama** (qwen3-embedding:latest).
- Cuando el usuario solicita auto-clasificación, se genera un embedding del título y contenido de la entrada y se calcula la **similitud coseno** contra los embeddings de sus temas mediante **pgvector**.
- Si la similitud supera un umbral configurable, el tema se asigna automáticamente.
- Este proceso es **síncrono** (la API espera la respuesta de Ollama) a diferencia de los resúmenes y transcripciones.
- La autoclasificación se lanza siempre que se haya modificado el título por defecto de la entrada, que el selector de topics de la entrada esté en AUTO, y que se realice un auto-guardado (debounce 1,8s)

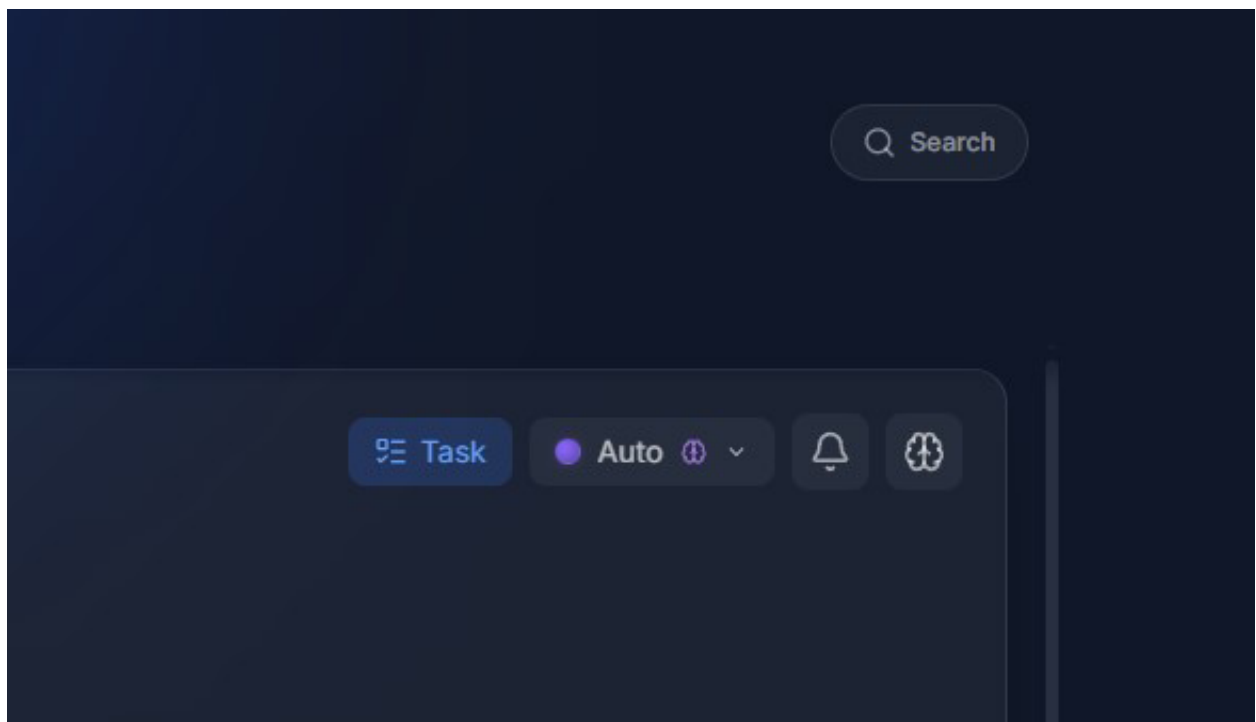


Figura 4.7.1 — Selección de auto en desplegable topics

4.8 OCR (Extracción de texto de imágenes)

- Al insertar un imagen aparecen botones de **scan** y **describe**.
- Permite **extraer texto de la imagen** adjunta o insertada en la entrada.
- Utiliza **Ollama** con un modelo de visión (glm-ocr:q8_0) para el reconocimiento óptico de caracteres.
- El proceso es síncrono y el texto extraído se inserta directamente en la entrada.
- Cola por usuario en el propio cliente para no lanzar mas de una petición a la vez, aunque por lo experimentado esta función en producción debería ser implementada con una cola en redis+worker al igual que otros procesos de IA.
- Protegido por los mismos guardrails de IA que el resto de funcionalidades.

NOTA: Para producción sería mejor opción utilizar un servicio externo orquestado de manera asíncrona mediante un worker y n8n al igual que los resúmenes, debido a que la carga de cpu al ejecutar el modelo en un vps es muy significativa al no disponer de gpu y memoria vram, lo que hace el proceso costoso y lento, se ha mantenido así por propósitos de experimentación para el proyecto TFM

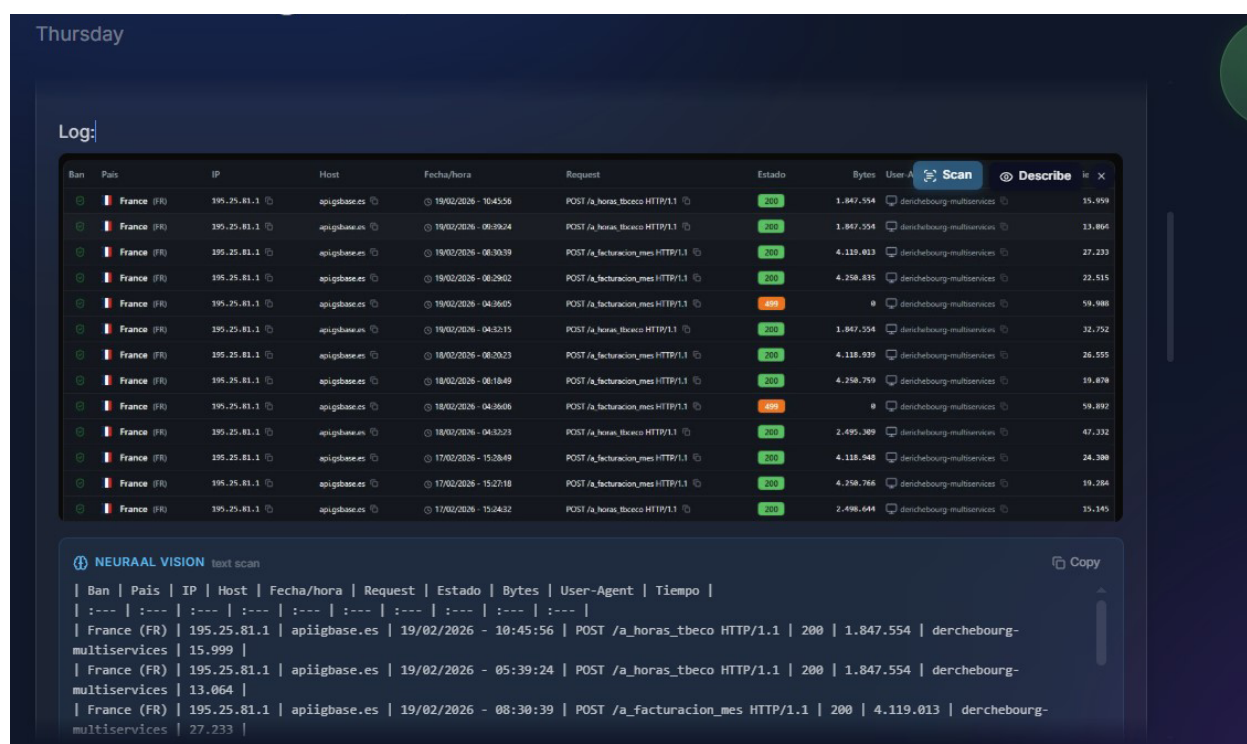


Figura 4.8.1 — Extracción de texto de una imagen

4.9 Transcripción de vídeos de YouTube

- Al incrustar un vídeo de YouTube en una entrada, aparece un **botón de transcripción**.
- Al pulsar, se lanza un flujo asíncrono similar al de resúmenes: BullMQ → n8n → API de transcripción → callback HMAC → notificación in-app.
- La transcripción completa se almacena asociada a la entrada.

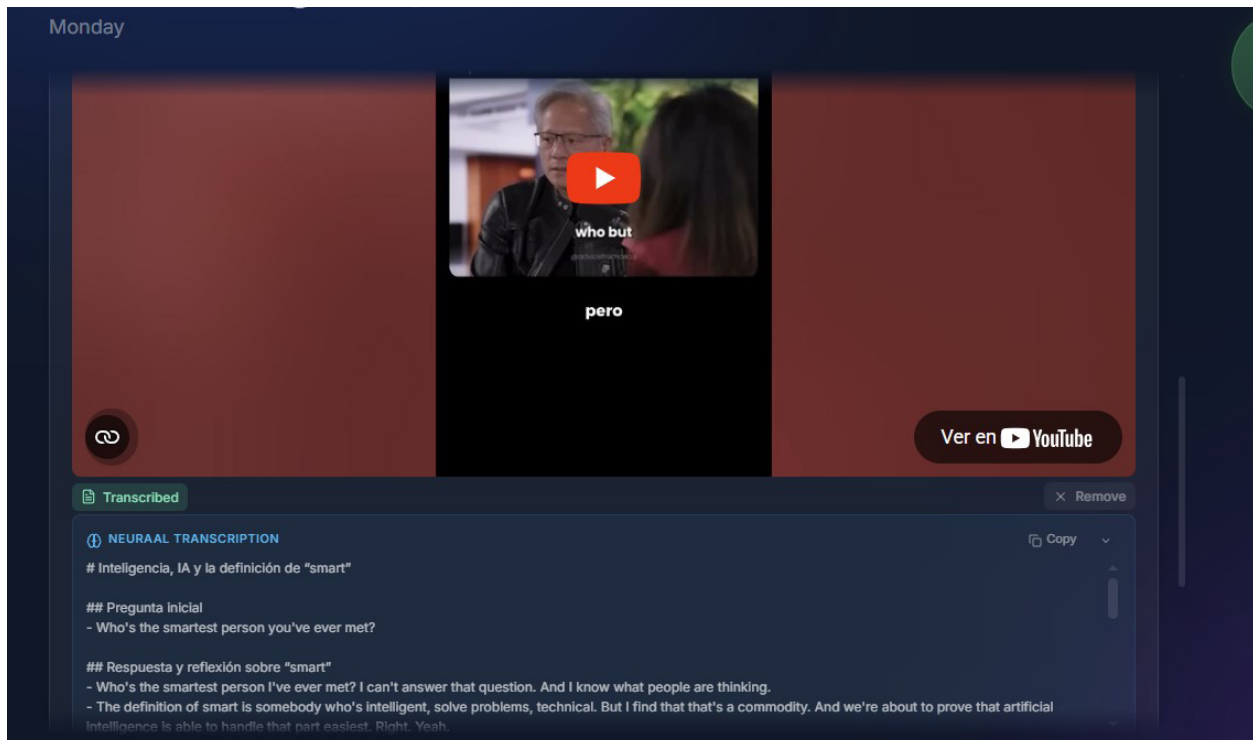


Figura 4.9.1 — Transcripción de video de Youtube

4.10 Adjuntos de archivos (File Attachments)

- Los usuarios pueden **adjuntar archivos** (imágenes, documentos) a cualquier entrada.
- El flujo utiliza **URLs firmadas de S3** (MinIO en desarrollo y producción, fácil migración S3/R2 para escalar):
 - La API genera una URL de subida firmada.
 - El frontend sube directamente al almacenamiento, sin pasar por el servidor.
 - Se confirma la subida y se registra el adjunto en la base de datos.
- Sistema de **cuotas**: límite por entrada (20 MB cada una) y límite global por usuario (1 GB).
- El panel de adjuntos muestra vista previa, tamaño y permite la descarga.

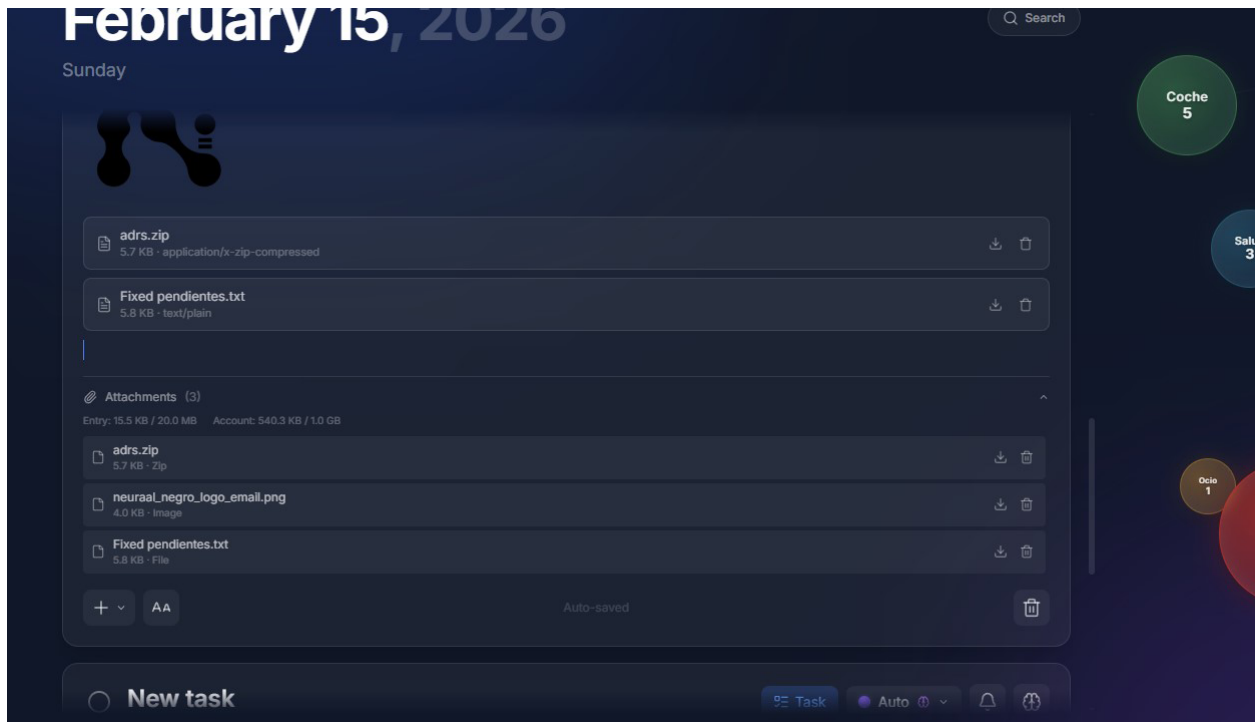


Figura 4.10.1 — Sección de adjuntos

4.11 Resumen semanal (Weekly Recap)

- Sección de **analítica semanal** con tres tipos de gráficos:
 - **Donut chart:** porcentaje de tareas completadas en la semana.
 - **Bar chart diario:** distribución de tareas/notas por día.
 - **Bubble chart de temas:** visualización de la actividad por tema.
- Generado a partir de los datos reales del usuario, utilizando **Recharts** y **D3.js**.



Figura 4.11.1 — Vista general de resumen semanal

4.12 Notas adhesivas (Stickies)

- Sistema de notas rápidas **persistentes independientes** del calendario.
- Layout **kanban** de dos columnas con soporte para drag-and-drop.
- Editor TipTap simplificado para el contenido de cada sticky.
- Reordenación por arrastre.

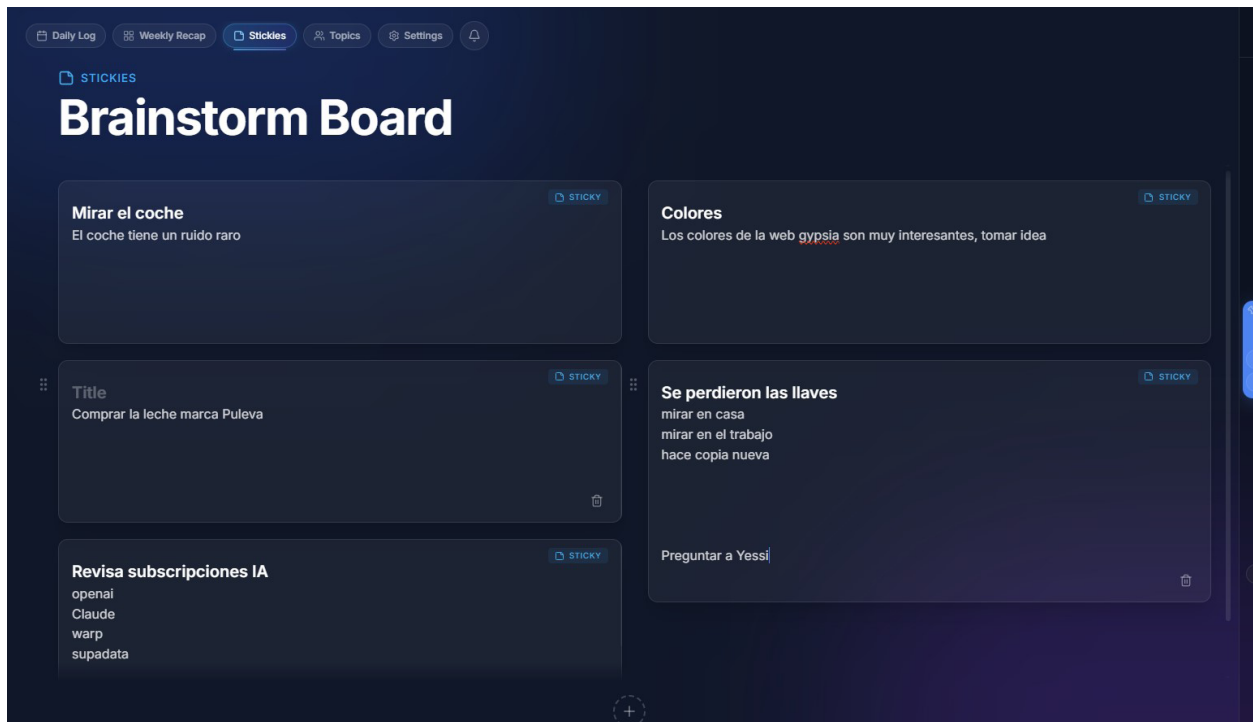


Figura 4.12.1 — Notas adhesivas

4.13 Barra de búsqueda global

- **Búsqueda en tiempo real** por título o contenido de todas las entradas del usuario.
- Extrae los datos almacenados localmente por el query de entradas mensuales del dashboard, lo que la hace muy rápida y sin peticiones extra al servidor.
- Navegación por teclado en los resultados de búsqueda.
- Al seleccionar un resultado, navega automáticamente al día correspondiente y hace scroll hasta la entrada.

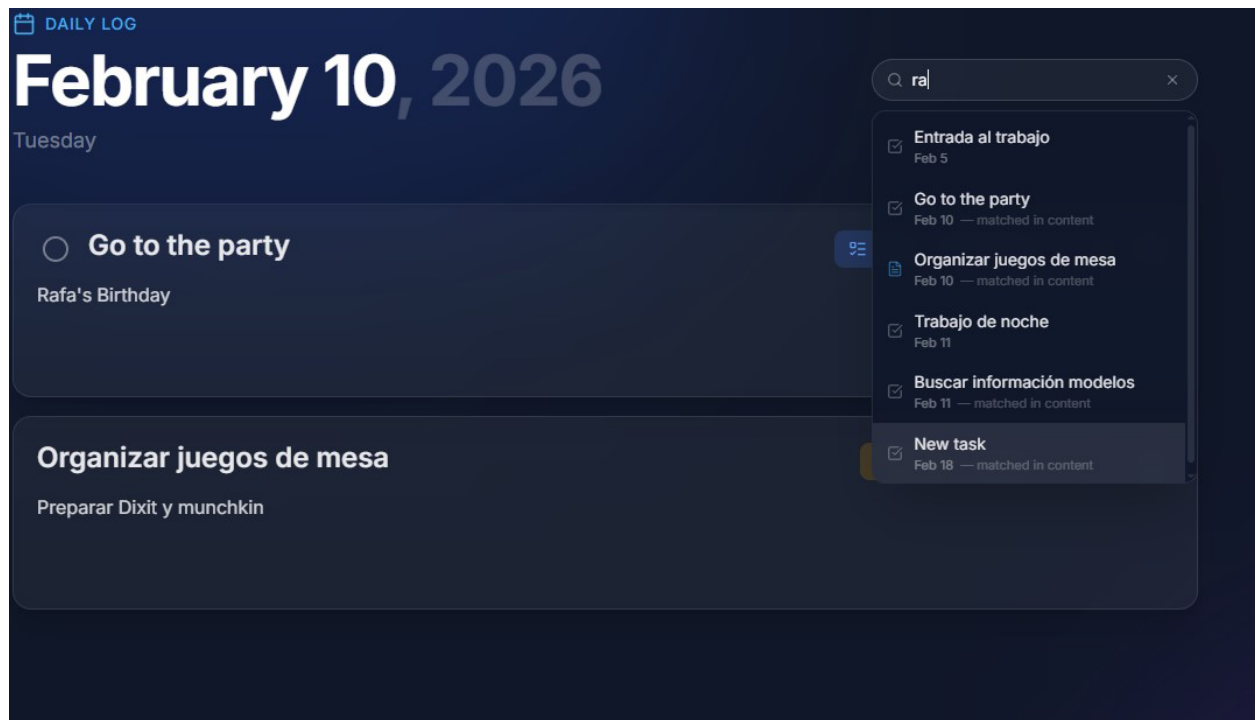


Figura 4.13.1 — Barra de búsqueda

4.14 Sistema de notificaciones in-app

- Centro de notificaciones que muestra el resultado de operaciones asíncronas:
 - REMINDER_SENT / REMINDER_FAILED
 - SUMMARY_DONE / SUMMARY_FAILED
 - TRANSCRIPTION_DONE / TRANSCRIPTION_FAILED
- **Watchers automáticos** que invalidan las queries de TanStack Query cuando llegan nuevas notificaciones, manteniendo la UI siempre sincronizada.

5. Stack tecnológico y justificación

5.1 Frontend

Tecnología	Versión	Justificación
Next.js (App Router)	16.1	Framework React de referencia en la industria. Server Components por defecto, enrutamiento basado en archivos, API routes integradas. Permite mantener frontend y backend en un solo repositorio con despliegue unificado.
React	19.2	Librería de UI más extendida del ecosistema, con soporte maduro para Server Components y hooks. Gran comunidad y ecosistema de herramientas.
TypeScript	5.x	Tipado estático que reduce errores en tiempo de desarrollo, mejora la autocompletación y documentación del código, y facilita refactorizaciones seguras en un proyecto full-stack.
Tailwind CSS	4.x	Framework de utilidades CSS que permite desarrollo rápido de UI responsiva sin mantener archivos CSS separados. Elimina CSS no utilizado en producción (tree-shaking).
Zustand	5.0	Gestor de estado global minimalista y ligero (~1 KB). Se utiliza exclusivamente para estado de UI (fecha seleccionada, posiciones de temas, sección activa). Su API simple evita el boilerplate de Redux.
TanStack Query	5.90	Gestiona todo el estado de servidor (entries, topics, reminders, notifications). Proporciona caché automática, invalidación inteligente, refetch en foco, y watchers basados en notificaciones. Separa claramente el estado del servidor del estado de la UI.
TipTap 3	3.19	Editor de texto enriquecido sobre ProseMirror. Elegido por su arquitectura extensible, soporte de React, bloques de código con syntax highlighting, y la posibilidad de crear extensiones personalizadas (YouTube embed, image attachment).
Framer Motion	12.x	Librería de animaciones declarativa para React. Utilizada para drag-and-drop (Reorder API), transiciones entre secciones y micro-interacciones. Ofrece animaciones a 60fps sin manipulación manual del DOM.
Recharts + D3.js	3.7 / 3.x	Recharts para gráficos estándar (donut, barras) por su API declarativa. D3 (force, hierarchy) para visualizaciones avanzadas como el force layout de los temas flotantes y el bubble chart de temas.
date-fns	4.1	Librería funcional e inmutable para manipulación de fechas. Alternativa tree-shakeable a Moment.js, importando solo las funciones necesarias.

5.2 Backend

Tecnología	Versión	Justificación
Prisma	7.3	ORM type-safe con generación automática de tipos TypeScript desde el schema. Migraciones declarativas y control total del esquema de base de datos. Integración con pgvector mediante SQL raw para operaciones vectoriales.
PostgreSQL + pgvector	16	Base de datos relacional robusta y madura. La extensión pgvector permite almacenar y buscar embeddings vectoriales (similitud coseno) directamente en la base de datos, sin necesidad de un servicio vectorial externo como Pinecone o Weaviate.
Redis	7.x	Almacén en memoria utilizado para tres propósitos: colas de trabajo BullMQ, rate limiting por usuario/IP, y control de concurrencia de peticiones de IA.
BullMQ	5.67	Sistema de colas basado en Redis para procesamiento asíncrono fiable. Soporta reintentos con backoff exponencial, jobs programados (reminders), y monitoring vía Bull Board. Elegido por su madurez y fiabilidad frente a alternativas como Agenda o pg-boss.
n8n	—	Plataforma de automatización de workflows que orquesta los flujos asíncronos de IA (resúmenes, transcripciones, recordatorios). Permite diseñar workflows visuales sin acoplar la lógica de orquestación al código de la aplicación.
Ollama	—	Servidor de modelos de IA ejecutados en local. Permite generar embeddings (qwen3-embedding:latest, 4096 dimensiones) y realizar OCR (glm-ocr:q8_0) sin depender de APIs externas de pago (OpenAI, Google). Reduce costes y mantiene la privacidad de los datos del usuario. NOTA: Para servidores sin gpu, sería más recomendable usar un servicio externo para OCR por costes y rendimiento
jose	6.1	Librería para firma y verificación de JWT conforme a estándares (RFC 7519). Elegida por su ligereza y compatibilidad con edge runtimes, frente a jsonwebtoken que depende de módulos nativos de Node.js.
bcryptjs	3.0	Hashing de contraseñas con bcrypt (12 rounds). Implementación en JavaScript puro, sin dependencias nativas, lo que simplifica el build Docker.
pino	10.3	Logger estructurado de alto rendimiento que emite JSON en producción. Redacción automática de campos sensibles (password, token, secret, cookie). Propagación de request ID para trazabilidad.
prom-client	15.1	Cliente oficial de Prometheus para Node.js. Expone métricas HTTP (duración, contadores), métricas de jobs BullMQ y métricas de guardrails de IA en formato Prometheus, listas para scraping por Grafana.

Sentry	10.39	Plataforma de error tracking y performance monitoring. Instrumentación en cliente, servidor, edge y workers. Session replay para depuración de errores de UI.
LangSmith	—	Plataforma para observabilidad de LLMs integrada con n8n. Trazabilidad y versionado prompts, costes, uso de tokens y latencias

5.3 Infraestructura y DevOps

Tecnología	Justificación
Docker (multi-stage build)	Imagen de producción mínima (~200MB) con tres etapas: dependencias, build y runtime. Sin código fuente ni herramientas de desarrollo en la imagen final.
GitHub Actions	CI/CD integrado con el repositorio. Dos workflows: CI (lint + type-check + tests + E2E) y Build & Deploy (Docker build + push GHCR + deploy SSH).
GHCR (GitHub Container Registry)	Registro de imágenes Docker integrado con GitHub, sin coste adicional para repositorios privados.
Caddy	Reverse proxy con TLS automático (Let's Encrypt/ZeroSSL). Configuración mínima comparada con Nginx: una línea para proxy reverso con HTTPS automático.
MinIO	Almacenamiento de objetos compatible con S3, auto-hospedado. Permite desarrollo local idéntico a producción sin costes de AWS. En el deploy de este proyecto finalmente se continuó con MinIO

5.4 Calidad de código

Tecnología	Justificación
Vitest	Test runner nativo de TypeScript, ~10x más rápido que Jest gracias a la infraestructura de Vite. API compatible con Jest.
Testing Library	Testing de componentes React centrado en comportamiento del usuario, no en detalles de implementación.
Playwright	E2E testing con Chromium real. Más fiable que Cypress para aplicaciones con auth compleja y cookies httpOnly.
ESLint + SonarJS	Linting con reglas de calidad de SonarJS: complejidad cognitiva, condicionales anidados, código duplicado.
Prettier	Formateo automático consistente. Eliminación de debates de estilo en revisiones de código.
Commitlint + Husky	Commits convencionales ('feat:', 'fix:', 'chore:') validados en pre-commit. Garantizan un historial de git legible y automatizable.
OpenAPI 3.1	Especificación API como fuente de verdad. Tipos TypeScript generados automáticamente desde el spec, garantizando sincronización frontend-backend.

6. Arquitectura del sistema

6.1 Visión general

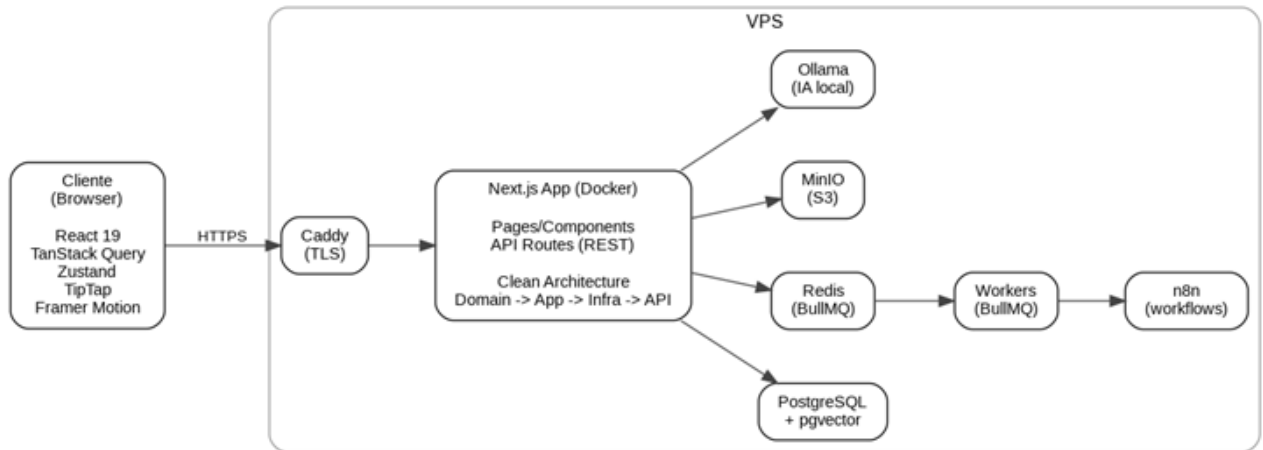


Figura 6.1.1 — Arquitectura de despliegue (alto nivel)

6.2 Clean Architecture (Backend)

El backend implementa **Clean Architecture** con cuatro capas y dirección de dependencias estricta:

Beneficios en este proyecto:

- **Testabilidad:** Los use cases se prueban con dobles de test (InMemory repos, Fake ports) sin necesidad de base de datos ni servicios externos.
- **Flexibilidad:** Se puede cambiar la implementación de cualquier puerto (por ejemplo, cambiar Ollama por OpenAI) sin tocar la lógica de negocio.
- **Separación de responsabilidades:** Cada capa tiene una responsabilidad clara; un cambio en la UI no afecta al dominio y viceversa.

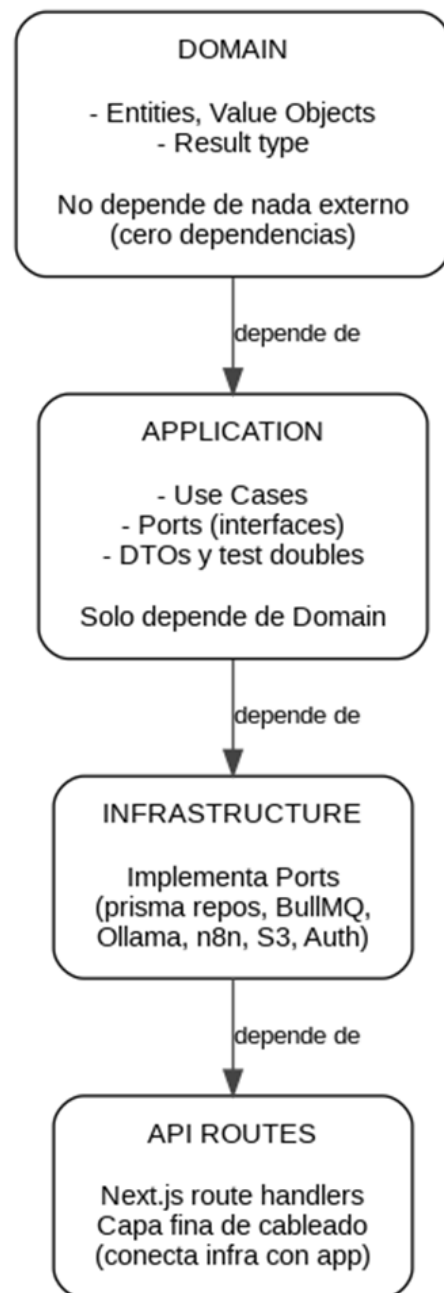


Figura 6.2.1 — Capas de Clean Architecture y dependencias

6.3 Arquitectura Frontend (Feature-Based)

El frontend organiza el código por funcionalidades aisladas:

```
src/  
  shared/      ← Alcance global (tipos, hooks, store, API client)  
  features/  
    dashboard/      ← Solo importa de shared/ y de sí misma  
    calendar/        ← Nunca importa de otra feature  
    tasks-container/  
    task-editor/  
    topics/  
    stickies/  
    weekly-recap/  
    notifications/  
    attachments/  
    settings/  
    layout/
```



Figura 6.3.1 — Estructura de carpetas (enfoque por features)

Regla de importación fundamental: Una feature **nunca** importa de otra feature. La comunicación entre features se realiza a través de `shared/` (estado global, query hooks).

7. Modelo de datos

El esquema de base de datos consta de **13 modelos** en Prisma:

Modelo	Descripción
User	Usuarios autenticados (email, password hash)
RefreshToken	Tokens de refresco (hash SHA-256, cadena de reemplazo para detección de reuso)
PasswordResetToken	Tokens de recuperación de contraseña (1h de expiración)
Topic	Temas/categorías definidos por el usuario (color, embedding vectorial)
Entry	Entradas del calendario (tareas o notas) con contenido JSON (TipTap), versión, orden
Reminder	Recordatorios programados (fecha/hora, estado, canal)
Notification	Notificaciones in-app (tipo, metadatos, leída/no leída)
Sticky	Notas adhesivas persistentes (contenido JSON, color, orden)
Attachment	Archivos adjuntos (clave S3, MIME type, tamaño, entry)
EntrySummaryRequest	Solicitudes de resumen (estado: pending/done/failed)
TranscriptionRequest	Solicitudes de transcripción (estado, URL YouTube)
AiUsageMonthly	Cuotas mensuales de IA por usuario y acción
AiUsageLedger	Registro detallado de cada petición de IA (auditoría)

Extensión pgvector: Los temas almacenan embeddings vectoriales de 4096 dimensiones en una columna vector(4096) gestionada mediante SQL raw, ya que Prisma no soporta nativamente tipos vectoriales.

8. Seguridad

La seguridad se ha tratado como un requisito por defecto, no como una capa añadida a posteriori. El diseño sigue las directrices del OWASP Top 10.

Autenticación JWT con rotación de tokens

Aspecto	Implementación
Access token	JWT HS256, httpOnly cookie, 60 min de vida.
Refresh token	Token aleatorio criptográfico, hash SHA-256 en BD, 30 días.
Rotación	Cada refresh emite un nuevo refresh token y revoca el anterior.
Detección de reuso	Si se reutiliza un token revocado, se revocan todos los tokens del usuario.
Rate limiting de login	5 intentos fallidos por IP → bloqueo de 5 minutos.
Política de contraseñas	8+ caracteres, mayúscula, minúscula, dígito, carácter especial, bcrypt 12 rounds.

Cabeceras de seguridad HTTP

Aplicadas a todas las rutas: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Strict-Transport-Security, Referrer-Policy, Permissions-Policy.

Webhooks con firma HMAC

Toda la comunicación bidireccional entre la aplicación y n8n está firmada con **HMAC-SHA256** + validación de timestamp (ventana de ± 5 minutos para prevenir ataques de replay).

Guardrails de IA

Protección multicapa contra abuso: rate limiting por minuto/hora (Redis), control de concurrencia por usuario, cuotas mensuales (PostgreSQL), y límite de tamaño de entrada.

Datos sensibles

- Logging estructurado con **redacción automática** de campos sensibles (password, token, secret, cookie).
- Variables de entorno gestionadas como secretos de GitHub Actions en CI/CD.
- Nunca se almacenan secretos en el código fuente.

9. Inteligencia Artificial integrada

Resumen de capacidades IA

Funcionalidad	Modelo	Tipo de proceso	Servicio
Resúmenes	LLM (vía n8n)	Asíncrono (202 → notificación)	n8n + LLM externo
Auto-clasificación	qwen3-embedding-latest	Síncrono	Ollama (local)
OCR	glm-ocr:q8_0	Síncrono	Ollama (local)
Transcripción	API externa (vía n8n)	Asíncrono (202 → notificación)	n8n + Supadata/Whisper

Flujo de auto-clasificación por embeddings

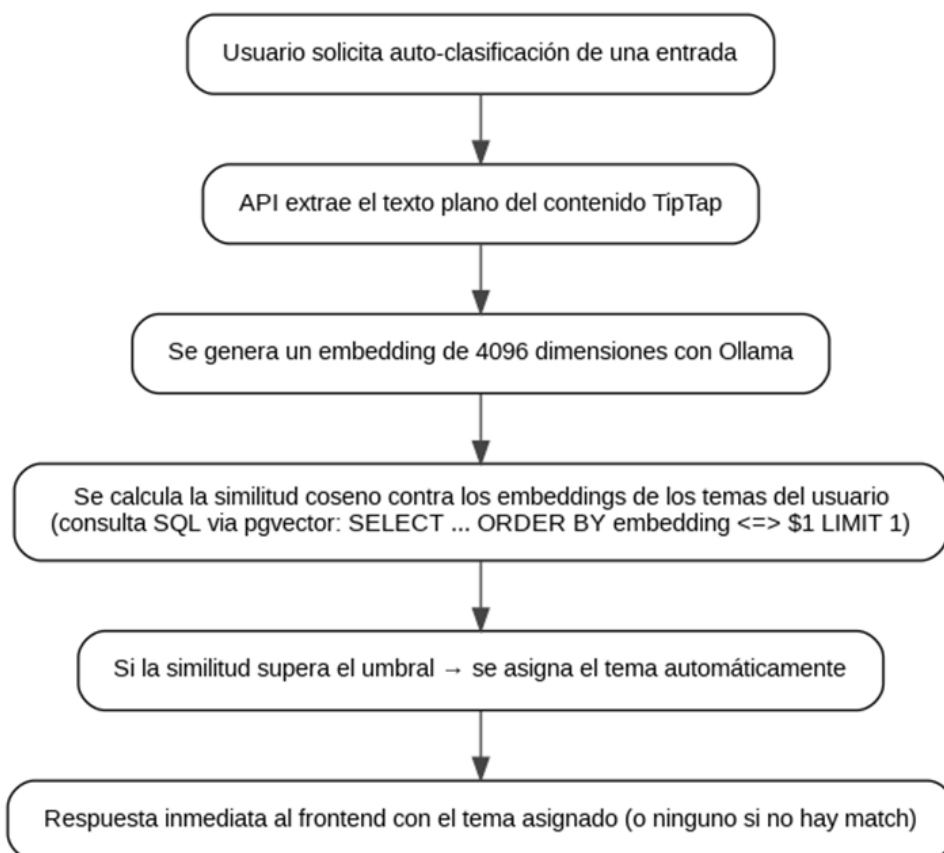


Figura 9.1.1 — Flujo de auto-clasificación por embeddings

Guardrails de IA

Todas las funcionalidades de IA están protegidas por un sistema centralizado:

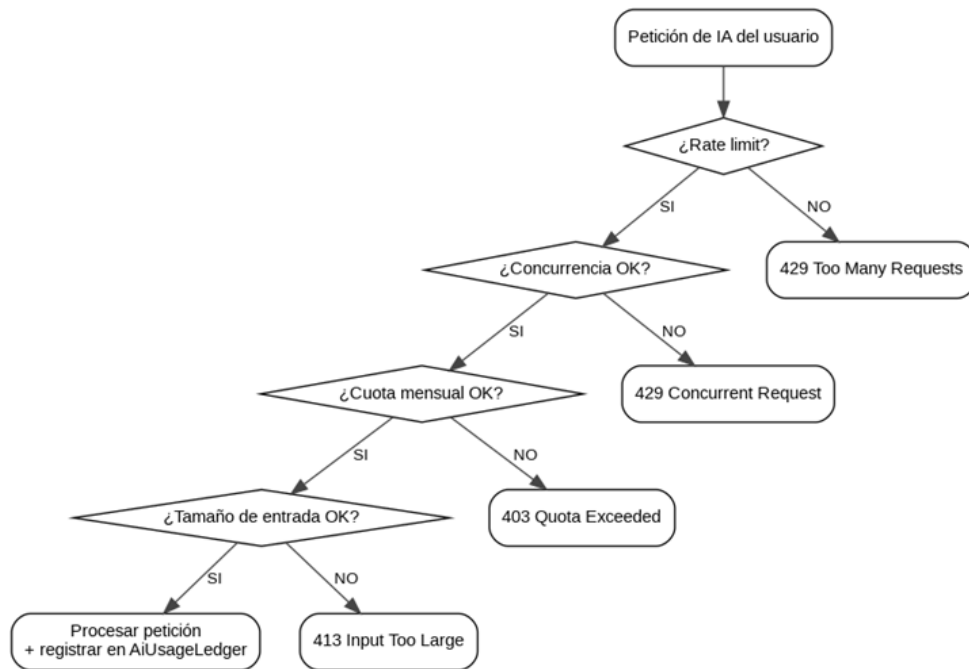


Figura 9.1.2 — Guardrails para peticiones de IA

El usuario puede consultar su consumo y cuotas restantes en la sección de **Configuración**.

10. Observabilidad y monitorización

Componente	Herramienta	Propósito
Error tracking	Sentry	Captura de errores en cliente, servidor, edge y workers. Session replay. Performance spans.
Métricas	Prometheus (prom-client)	Duración HTTP, contadores de peticiones, métricas de jobs BullMQ, guardrails de IA. Endpoint: `/api/metrics`.
Logging estructurado	pino	JSON en producción, pretty-print en desarrollo. Auto-redacción de campos sensibles. Request ID en cada petición.
Health check	`/api/health`	Estado de todas las dependencias: PostgreSQL, Redis, Ollama, n8n, S3. Respuesta estructurada con latencia por servicio.
Queue monitoring	Bull Board	UI web para monitorizar jobs de BullMQ: estado, reintentos, errores.
LLM tracking	LangSmith	Plataforma web para monitorizar y rastrear cada ejecución del agente de IA (resúmenes, transcripciones) con registro completo de entrada/salida, análisis de latencia, uso de tokens, seguimiento de costos y control de versiones de los avisos.

Health check response

```
{
  "status": "ok",
  "checks": {
    "db": { "ok": true, "latencyMs": 3 },
    "redis": { "ok": true, "latencyMs": 1 },
    "ollama": { "ok": true, "latencyMs": 45 },
    "n8n": { "ok": true, "latencyMs": 12 },
    "s3": { "ok": true, "latencyMs": 8 }
  }
}
```

11. Calidad del software

Estrategia de testing

Capa	Herramienta	Alcance
Unitarios	Vitest	Value objects, entities, utilidades puras
Integración	Vitest	Use cases con test doubles (InMemory repos, Fake ports)
API	Vitest	Route handlers con use cases mockeados
Componentes	Testing Library	Componentes React (comportamiento de usuario)
E2E	Playwright	Flujos críticos completos (auth, dashboard, salud)

Ficheros de test: 157+, ejecutados en CI en cada push.

Cobertura basada en riesgo

- **100%** para guardrails de IA (GuardAiAction, ConsumeAiRequest) y lógica de autenticación.
- **80%** para rutas de usuario y componentes clave.
- **0%** intencionado para tipos, constantes y código generado.

Calidad estática

- **ESLint + SonarJS:** Complejidad cognitiva (límite: 15), condicionales anidados, funciones idénticas.
- **Prettier:** Formateo automático uniforme.
- **Commitlint + Husky:** Validación de mensajes de commit convencionales en pre-commit.
- **lint-staged:** Solo se linteian los archivos modificados en el commit.

Especificación de API

- **OpenAPI 3.1** como fuente de verdad (openapi/spec.ts).
- Tipos TypeScript **generados automáticamente** desde la especificación (openapi-typescript).
- Esto garantiza que el frontend y el backend siempre comparten el mismo contrato de API.

12. Despliegue y CI/CD

Pipeline completo

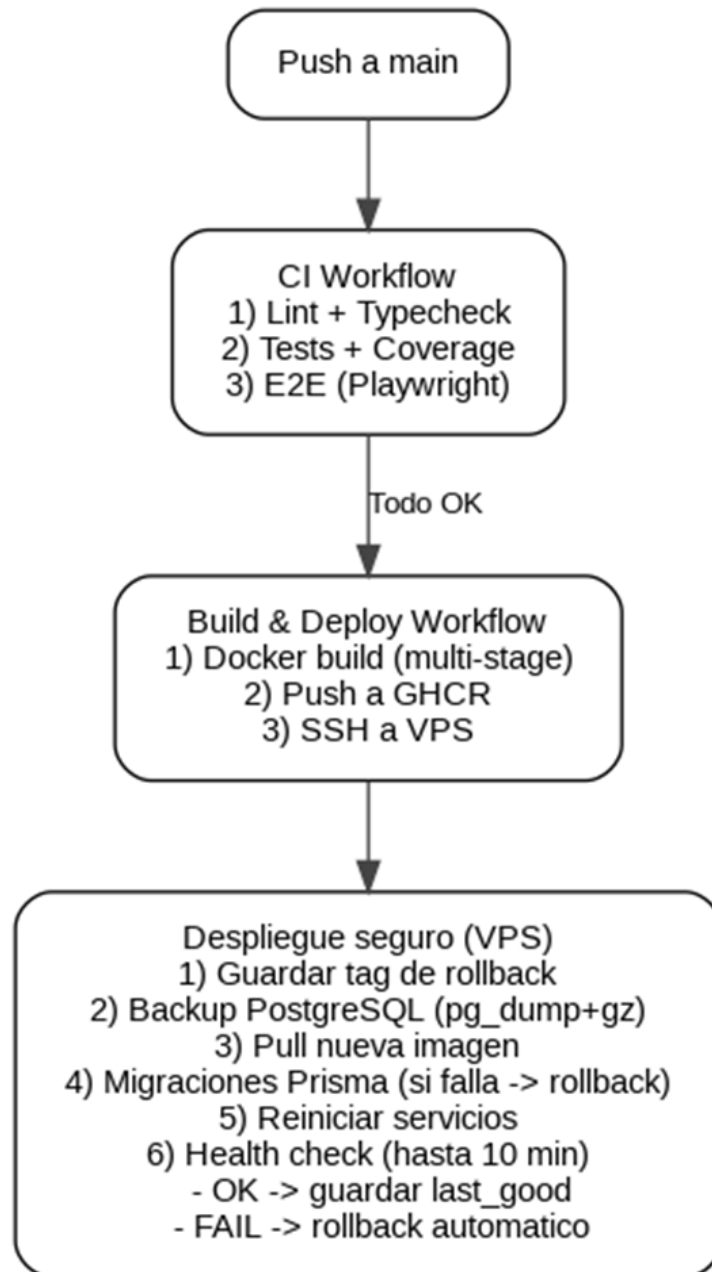


Figura 12.1.1 — Pipeline CI/CD y despliegue seguro

Infraestructura de producción

Servicio	Imagen	Función
Caddy	caddy:latest	Reverse proxy + TLS automático
App	ghcr.io/.../neuraal:sha	Aplicación Next.js
PostgreSQL	pgvector/pgvector:pg16	Base de datos + vectores
Redis	redis:7-alpine	Colas + rate limiting
MinIO	minio/minio	Almacenamiento S3
n8n	n8nio/n8n	Automatización de workflows
Ollama	ollama/ollama	Modelos de IA locales

Backup y recuperación

- **Backup automático** de PostgreSQL antes de cada despliegue (comprimido con gzip).
- Retención: últimos 7 backups; eliminación solo si >14 días y >7 backups.
- Procedimiento de **restauración manual** documentado para situaciones de emergencia (neuraal-deploy-rollback-backup-manual.md).

13. Reflexión, aprendizajes y dificultades

Este proyecto ha significado un antes y un después para mí. Ha sido el paso clave para empezar a evolucionar como desarrollador: me he enfrentado a tecnologías que no había usado nunca (o apenas conocía) y, sobre todo, he tenido que tomar decisiones reales de ingeniería, comparando alternativas, evaluando pros y contras y asumiendo los compromisos de cada elección.

Principales aprendizajes

- **Decidir con criterio:** entender que no existe “la mejor” tecnología en abstracto, sino la más adecuada para el contexto. El proceso de informarme, probar y escoger ha sido parte central del proyecto.
- **Arquitectura y mantenimiento:** trabajar con una estructura clara y separada por responsabilidades me ayudó a mantener el control del proyecto a medida que crecía.
- **Calidad y pruebas:** interioricé la importancia de probar la lógica clave y de apoyarme en herramientas de testing para reducir regresiones durante los cambios.
- **Operación y despliegue:** ha sido la primera web que publico “para el mundo”. Aprendí que no basta con que el código funcione en local: hay que pensar en despliegue, configuración, errores en producción y capacidad de recuperación.

Dificultades encontradas

El proceso no ha sido sencillo. Al ser la primera vez que abordaba varias de estas piezas, tenía que investigar pros y contras antes de implementar una cosa u otra, hacer pruebas, implementaciones que al final no funcionan como se esperaba, cosas que hay que ir descartando, lo típico de un proyecto real.

Además, el plazo fue especialmente exigente: tuve que compaginar el desarrollo con mi trabajo y, por el límite temporal del marco FUNDAE, el proyecto se convirtió en un reto intenso. Ha sido un proceso en el que he disfrutado y sufrido, pero terminar a tiempo y con una aplicación funcional me deja con una sensación increíble de logro.

Impacto personal

Más allá de lo técnico, este TFM me ha aportado confianza. Me ha hecho ver todo lo que he aprendido y, a la vez, la cantidad de cosas que me quedan por aprender aún. Pero, por primera vez en mucho tiempo, siento que estoy “en el camino”: que nunca es tarde, que puedo avanzar y que voy a ser capaz de seguir creciendo como programador.

14. Líneas futuras

El proyecto tiene un alcance completo como MVP, pero existen varias líneas de evolución que mejorarían significativamente la experiencia del usuario:

- PWA (Progressive Web App): Convertir la aplicación en una PWA permitiría instalación en el dispositivo móvil, notificaciones push nativas y uso offline parcial.
- Colaboración multiusuario: Compartir temas, entradas y listas de tareas entre usuarios. Esto requeriría un modelo de permisos (owner/editor/viewer) y sincronización en tiempo real (WebSockets).
- Mejora de la analítica: Ampliar el Weekly Recap con tendencias mensuales, heatmaps de actividad y sugerencias basadas en patrones del usuario.
- Internacionalización (i18n): Actualmente la UI es en inglés; añadir soporte multiidioma con next-intl o similar.
- Reconciliar n8n: Actualmente si por algún motivo el workflow de n8n falla, no hay manera de detectarlo y el usuario queda con la solicitud pendiente, en la implementación actual se utiliza un TTL de 10 min para que ignore este tipo de solicitudes que han quedado por algún motivo sin resolver, pero no es la solución óptima, sería mejor implementar un reconciler que buscase este tipo de solicitudes en DB para reintentarlas o eliminarlas.
- Buscador General: Ahora mismo el buscador solo busca en el mes actual mirando datos almacenados localmente, a futuro se podría implementar un buscador general que buscase en todas las entradas del año.
- Panel de control Admin: Implementar un panel de control donde poder ver estadísticas de usuarios, poder desloguear usuarios, o incluso banear (implementando sistema de estados/rango de usuarios)
- Chatbot interactivo e inteligente: Chatbot con el que poder crear tareas, aplazar o dar por completadas, interactividad via whatsapp y via web.
- Aginacion masiva de tareas: Utilidad para asignar tareas repetitivas masivamente a los días seleccionados (por día de mes o por día semana)
- Mejorar la experiencia UI: Sobre todo en mobile
- Tests de rendimiento: Implementar pruebas de carga para validar el comportamiento bajo concurrencia (k6 o Artillery).

15. Conclusiones

Neuraal nace como respuesta a un problema real: la necesidad de capturar ideas con la misma rapidez que en una app sencilla, sin renunciar a una organización más potente cuando el proyecto o la información crecen.

A nivel de resultados, el proyecto demuestra que, cuidando el proceso de toma de decisiones y buenas prácticas, es posible construir una aplicación web de productividad con una experiencia visual diferenciada, preparada para evolucionar con funcionalidades avanzadas (incluida la IA) y con un enfoque profesional en aspectos transversales como seguridad, pruebas y despliegue.

Como conclusión final, este TFM ha sido especialmente valioso por dos motivos:

- Producto: se ha materializado una aplicación real, desplegada y utilizable.
- Proceso: el aprendizaje obtenido (técnico y personal) ha sido el mayor retorno del proyecto, porque me deja una base sólida para seguir desarrollando, iterar sobre Neuraal y afrontar proyectos más complejos con más criterio y confianza.

Anexo: Architecture Decision Records (ADRs)

#	Decisión	Estado
001	Next.js App Router + estructura basada en features	Aceptada
002	State management: Zustand (UI) + TanStack Query (servidor)	Aceptada
003	Testing: Vitest + Testing Library + Playwright	Aceptada
004	Auth: JWT access/refresh en httpOnly cookies	Aceptada
005	Observabilidad con Sentry	Aceptada
006	Auth: OAuth + Auth.js *(reemplazada por ADR-004)*	Reemplazada
007	Persistencia híbrida: PostgreSQL + S3	Aceptada
008	Automatización: n8n + BullMQ	Aceptada
009	pgvector embeddings para auto-clasificación de temas	Aceptada
010	OpenAPI spec como fuente de verdad + tipos generados	Aceptada
011	Guardrails de IA y tracking de uso	Aceptada
012	Editor de texto enriquecido: TipTap 3	Aceptada
013	WhatsApp vía Evolution API *(bloqueado por Meta)*	Rechazada
014	Docker multi-stage + pipeline CI/CD	Aceptada
015	Logging estructurado (pino) + métricas Prometheus	Aceptada
016	TanStack Query para estado del servidor	Aceptada
017	Viabilidad de Ollama en producción vs servicios IA externos	Aceptada

